LIVE PROJECT REPORT

"AI Model for Early Detection of Diabetes using lifecycle and medical data - @HealthWise".

Submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Technology
Computer Science and Engineering

Supervisor:
Supervisor Name: Ms. Preeti
Designation: Assistant Professor

Submitted by:
Student Name: CHIRAG
Roll No: 10323210081



SRM UNIVERSITY, DELHI-NCR, SONEPAT, HARYANA
Plot No. 39, Rajiv Gandhi Education City, Sonepat, Haryana 131029

MAY 2025

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING/DEPARTMENT OF COMPUTER SCIENCE

PROJECT APPROVAL PAGE

B.Tech CSE [Core]

GROUP No: SESSION: 2024-2025

To be filled in by the student: -

Student Name(s): **CHIRAG** Enrollment Number: **10323210081** Project Title **AI Model for Early Detection of Diabetes using lifecycle and medical data- @HealthWise**

Signature **……**

To be filled in by the supervisor: -

This is to certify that the project submitted and presented by the above-mentioned student(s) is

COMPLETE ACCEPTED

and the draft of the graduation project report has been corrected for all content flaws, typing errors, and language mistakes.

Supervisor name and Signature: Date:

Examiner's Name and Signature: Date:

HOD's Signature: Date:

*NOTE: The student may proceed with Ring binding ONLY after this section has been completed. The student shall submit three copies of Ring bound draft report for the Minor/Major Project to the panel of examiners on the day of the final presentation.

For Departmental Office use only

Date received:

Signatures:

TABLE OF CONTENTS

CANDIDATE'S DECLARATION v
CERTIFICATE vi
ACKNOWLEGDEMENT..... vii
LIST OF FIGURES viii
LIST OF TABLES..... ix

Chapter 1: Introduction..... 10-13

1.1 Project Title - AI Model for Early Detection of Diabetes using lifecycle and medical data – "@HealthWise"..... 10
1.2 Problem Statement..... 10
1.3 Objective..... 11
1.4 Scope..... 11
1.5 Motivation..... 12
1.6 Significance of the Study..... 12

Chapter 2: Methodology..... 14-28

2.1 Dataset Description..... 14
2.2 Data Cleaning and Preprocessing..... 14
2.2.1 Handling Missing Values..... 14
2.2.2 Feature Correlation..... 15
2.3 Feature Scaling and Train-Test Split..... 16
2.4 Class Imbalance Handling..... 17
2.5 Model Selection and Training..... 18
2.5.1 RandomForestClassifier..... 18
2.5.2 Hyperparameter Tuning..... 18
2.6 Model Evaluation..... 19
2.7 Feature Importance Analysis..... 22
2.8 Why Random Forest Over XGBoost?..... 23
2.9 Prediction Function..... 24
2.10 Tools and Technologies Used..... 24
2.10.1 Python-Based Backend Tools and Libraries..... 24
2.10.2 Frontend Technology Stack..... 25

2.10.3 Integration Workflow……..…28
Chapter 3: System Design……..29-35
3.1 Architecture Overview……29
3.2 Functional Requirements……30
3.3 Non-Functional Requirements……30
3.4 User Interface Design……..…31
3.5 Data Flow & Flowchart Explanation…….....31
3.6 Data Models of the System…….34
3.7 Module Interaction and Integration……35
3.8 Security Considerations……..35
Chapter 4: Implementation……..36-50
4.1 Frontend (React.js)……..36
4.2 Backend (Flask)……...44
4.2.1 app.py – Flask Application Setup…….....44
4.2.2 routes.py – API Endpoints……45
4.3 Model Integration……....46
4.4 Logic for Clinical Estimation……..46
4.5 Code Structure (Project Overview)…….48
4.6 Coding Challenges and Solutions……50
4.7 Data Flow Summary……50
4.8 Module Interaction Summary……..50
Chapter 5: Testing and Results……..51-58
5.1 Testing Methodologies……51
5.2 Model Testing and Evaluation…….....51
5.3 Performance Metrics (RandomForestClassifier)…….52
5.4 Result Presentation……..52
Chapter 6: Conclusion and Future Work……..59-63
6.1 Conclusion……..59
6.2 Summary of Findings…….59
6.3 Limitations……..……60
6.4 Future Enhancements……..……61
References……..64

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project entitled "**AI Model for Early Detection of Diabetes using lifecycle and medical data**" in partial fulfillment of the requirement for the award of the Degree of Bachelor of Technology in Computer Science and Engineering SRM University, Delhi-NCR, Sonepat, Haryana, (India) is an authentic record of our own work carried out under the supervision of **Ms. Preeti** as Live project in 4th Semester during the academic year 2024-25. The matter presented in this project has been submitted for the semester exam of this University.

STUDENT 1 - CHIRAG

Registration No: 10323210081

CERTIFICATE

This is to certify that "**AI Model for Early Detection of Diabetes using lifecycle and medical data- @HealthWise**" project is a Bonafide work completed as a Live Project (B.Tech CSE CORE- 23CS0204) in partial fulfilment for the award of the degree of Bachelor of Technology in CSE under the guidance of **Ms. Preeti**. This project is submitted by Student1(CHIRAG - 10323210081), during the academic Semester IV of year 2024-2025 of SRM University, Delhi-NCR, Sonipat, Haryana.

Superviser Name:- Ms. Preeti

Designation:- Assistant Professor

ACKNOWLEGDEMENT

Most importantly, we would like to express our immense gratitude to our supervisor **Ms. Preeti,** for her patient guidance, encouragement, training, and advice he/she has provided throughout the time. The knowledge we have gained throughout this time will stay with us for years to come. We have been extremely lucky to have such a supervisor who cared for our work, and responded to our questions and queries so promptly. It was a great privilege to get to collaborate with him/her and work under his/her guidance. Though the following project and report is an individual work, we would never be able to explore the depths of the topic, without the help, support, guidance & efforts of our able supervisor. His/Her infectious enthusiasm and unlimited zeal have been major driving forces throughout the work.

We would like to extend our indebtedness to the **Project Committee Members**, Department of Computer Science and Engineering/ Department of Computer Science, SRM University Delhi-NCR, Sonepat as we have benefited so much from the constructive criticism and suggestions given to us during the project.

We feel compelled to articulate our thankfulness to **Prof. M. Mohan**, HOD, Department of Computer Science and Engineering, SRM University for his encouragement which was a source of inspiration.

Lastly, we are indebted to all the teaching and non-teaching staff members of the university for helping us directly or indirectly by all means throughout the course of our study and project work.

LIST OF FIGURES

Figure-2.2.1 Handling Missing or Invalid Medical Data……...15
Figure-2.2.2 Statistical Correlation Between Medical Attributes……15
Figure-2.2.3 Heatmap of Feature Correlations……..16
Figure-2.3.1 Feature Scaling and Splitting Dataset into Train and Test……17
Figure-2.3.2 Transformed Training Features After Standardization……17
Figure-2.4 Balancing the Dataset Using SMOTE……18
Figure-2.5.1 Training the Random Forest Classifier Model……..18
Figure-2.5.2 Hyperparameter Tuning using GridSearchCV……19
Figure-2.6.1 Model Performance Metrics……20
Figure-2.6.2 Confusion Matrix and Classification Metrics Report……...21
Figure-2.7.1 Feature Importance Visualization……22
Figure-2.7.2 Bar Plot Showing Feature Importance……23
Figure-2.9 Python Function for Real-Time Diabetes Risk Prediction……..24
Figure-3.5 End-to-End System Architecture Flowchart……34
Figure-4.4.1 Route-Aware Header Component (Header.jsx)……37
Figure-4.1.2 Structured Footer Component (Footer.jsx)……..39
Figure-4.1.3 Homepage Component (HomePage.jsx)……..41
Figure-4.1.4 Health and Lifestyle Quiz Questions……43
Figure-4.2.1 Flask Application Entry Point for HealthWise ML API……...44
Figure-4.2.2 Diabetes Prediction API Endpoint Using Flask Blueprint……45
Figure-4.3 Model Loading for Diabetes Prediction……..46
Figure-4.4 Mapping Lifestyle Inputs to Clinical Features in quizToClinical.js……47
Figure-4.5 Code structure of the project @HealthWise……49
Figure-5.4.1 Home Page and FAQ Section of the @HealthWise……54
Figure-5.4.2 Quiz Submission Form @HealthWise……55
Figure-5.4.3 Dashboard Page @HealthWise……56
Figure-5.4.3.1 History Page @HealthWise……...57
Figure-5.4.3.2 Detailed History Table @HealthWise……..58

LIST OF TABLES

Table-2.1 Feature Descriptions of the Diabetes Dataset……..14

Table-2.6 Evaluation Metrics for Model Performance…….19

Table-2.10.1 Backend Libraries and Their Purpose……..25

Table-2.10.2 Frontend Technology Stack and Justification…….……25

Table-3.3 Non-Functional Requirements…….30

Table-3.4 User Interface Design Flow…….……31

Table-3.6 Overview of Data Models ……..……..……..34

Table-3.7 Module Interaction and Integration Map…….……35

Table-4.8 Summary of Module Roles and Interaction……..……50

Table-6.3 Technical and Functional Limitations of @HealthWise……...60

Table-6.4 Feature Enhancements and Functional Improvements Roadmap …….61

Chapter 1: Introduction

1.1 Project Title - AI Model for Early Detection of Diabetes using lifecycle and medical data – "HealthWise"

Diabetes Mellitus (DM) is a chronic metabolic disorder marked by elevated levels of blood glucose due to defects in insulin production, insulin action, or both. According to the World Health Organization (WHO), diabetes is one of the top ten causes of mortality worldwide, with an alarming increase in prevalence, particularly in low- and middle-income countries. The two primary forms of the disease are:

- **Type 1 Diabetes**: An autoimmune condition where the immune system destroys insulin-producing beta cells in the pancreas, usually occurring in childhood or adolescence.
- **Type 2 Diabetes**: A more prevalent form often associated with lifestyle factors such as poor diet, physical inactivity, obesity, and aging. It is characterized by insulin resistance and relative insulin deficiency.

Globally, the growing burden of Type 2 diabetes is a major public health concern. It is often diagnosed at a late stage, after the onset of complications such as cardiovascular diseases, neuropathy, nephropathy, and retinopathy. These complications not only degrade the quality of life but also contribute significantly to healthcare costs and mortality rates.

Traditional methods of diagnosing diabetes involve blood glucose testing, HbA1c analysis, and oral glucose tolerance tests. While accurate, these methods require laboratory access, trained personnel, and are often not feasible for routine screening in resource-limited or rural areas.

In recent years, Artificial Intelligence (AI) has emerged as a promising tool in healthcare for disease detection, prognosis prediction, and personalized medicine. AI models, when trained on large, relevant datasets, can learn complex patterns in data and provide rapid, accurate predictions. This capability opens doors for accessible, scalable diagnostic tools that can function even without lab-based clinical parameters.

1.2 Problem Statement

Despite the increasing prevalence of Type 2 diabetes, a significant proportion of affected individuals remain undiagnosed until the disease has progressed to a serious stage. The reasons for this include:

- Lack of awareness of early symptoms.
- Limited access to regular healthcare facilities.
- High cost and logistical barriers of conventional testing.
- Sociocultural stigmas associated with hospital visits or disease acknowledgment.

The absence of a simple, accessible, and cost-effective screening tool is a critical gap in diabetes management and prevention. Hence, there is a pressing need for a non-invasive, AI-based solution that uses easily obtainable lifestyle and demographic inputs to assess an individual's diabetes risk before severe complications develop.

1.3 Objective

This research aims to address the early detection gap by developing a comprehensive, AI-driven diabetes prediction system. The key objectives are:

- To design a **lifestyle-based quiz** that gathers user inputs like physical activity, diet, smoking habits, sleep patterns, stress levels, and demographics (e.g., age, gender).
- To map these lifestyle indicators to inferred clinical features (e.g., glucose levels, BMI, insulin sensitivity) through intelligent estimation models.
- To develop and train AI models—using publicly available datasets such as the Pima Indian Diabetes Dataset and the Lakh MIDI Dataset—to predict diabetes status based on combined inferred and real inputs.
- To integrate the model into an **intuitive web-based application** that offers real-time risk assessment and personalized health recommendations.
- To ensure the solution is lightweight, resource-efficient, and scalab**le** for deployment across various platforms, including mobile devices.

1.4 Scope

The study encompasses the following components:

- **Data Collection and Preprocessing**: Gathering clinical and lifestyle data from reliable sources, cleaning it, and preparing it for model training.
- **Quiz Design and Feature Mapping**: Constructing a comprehensive questionnaire to capture user behavior, and devising rules or models to infer clinical equivalents.
- **Model Development and Evaluation**: Building and validating machine learning

models to ensure high accuracy, sensitivity, and specificity.

- **Web Application Development**: Designing a user-friendly front end that presents predictions, interpretations, and recommendations in an understandable format.
- **Deployment and Optimization**: Hosting the system using minimal computational resources while ensuring reliability and speed.

This research does not aim to replace medical diagnosis but rather to assist in early risk identification, particularly in settings where traditional screening is impractical.

1.5 Motivation

The motivation behind this study stems from the recognition that **Type 2 diabetes is largely preventable and manageable** when identified early. According to numerous studies, lifestyle interventions—such as increased physical activity, dietary changes, and weight management—can significantly reduce the risk of developing diabetes.

However, for such interventions to be effective, timely identification of at-risk individuals is critical. By leveraging AI to analyze lifestyle data, we can reach underserved populations and promote proactive health behavior without necessitating clinical tests upfront.

Moreover, the COVID-19 pandemic has underscored the importance of remote health monitoring tools. The proposed system aligns with this global shift towards digital, self-service healthcare solutions.

1.6 Significance of the Study

This study contributes meaningfully to the transformation of healthcare through the following aspects:

- **Democratization of Health Risk Assessment**: Makes diabetes screening accessible to individuals without requiring medical infrastructure or in-person consultations.
- **Promotion of Preventive Healthcare**: Encourages individuals to understand and improve their health status before complications arise.
- **Cost-Effective Solution**: Reduces the economic burden of frequent clinical visits and laboratory tests, especially in economically challenged regions.
- **Support for Healthcare Professionals**: Offers a preliminary assessment tool that can help prioritize patient care in clinical settings.

- **Advancement in AI for Public Health**: Demonstrates the power of AI not just for treatment support but also for community-level disease prevention.

By focusing on awareness, accessibility, and action, this research seeks to empower users with knowledge and tools to take control of their health and reduce the global burden of diabetes.

Chapter 2: Methodology

This chapter outlines the systematic approach employed to develop a machine learning-based diabetes prediction system. It includes dataset handling, preprocessing, modeling, and evaluation phases, based directly on the code implemented in Python using industry-standard libraries.

2.1 Dataset Description

The dataset used for this project is the Pima Indian Diabetes Dataset (PIDD), sourced from Kaggle, containing 768 records and 8 input features, along with a binary outcome variable. These features include both physiological measurements and demographic indicators.

Feature	Description
Pregnancies	Number of times the individual has been pregnant
Glucose	Plasma glucose concentration a few hours after fasting
BloodPressure	Diastolic blood pressure (mm Hg)
SkinThickness	Triceps skin fold thickness (mm)
Insulin	2-hour serum insulin (mu U/ml)
BMI	Body Mass Index
DiabetesPedigree	Genetic predisposition score
Age	Age in years
Outcome	1 = Diabetic, 0 = Non-diabetic

Table-2.1 *Feature Descriptions of the Diabetes Dataset*

2.2 Data Cleaning and Preprocessing

2.2.1 Handling Missing Values

In the original dataset, many physiological variables such as Glucose, BloodPressure, SkinThickness, Insulin, and BMI contained zero values, which are clinically implausible and indicative of missing data.

To handle this:

- All zero entries in these columns were replaced with NaN.
- The median of each column was calculated and used to impute these missing values, preserving data distribution and reducing outlier sensitivity.

```
# Replace zeroes with NaN in important medical features
columns_to_replace = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
diabetes_dataset[columns_to_replace] = diabetes_dataset[columns_to_replace].replace(0, np.nan)

# Fill NaNs with median values
diabetes_dataset.fillna(diabetes_dataset.median(), inplace=True)
```

Figure-2.2.1: *Code Snippet of Handling Missing or Invalid Medical Data Using Median Imputation*

2.2.2 Feature Correlation

A heatmap was generated to inspect correlations between features. This helped in understanding the interdependency and redundancy among features before model training.

```
correlation = diabetes_dataset.corr()
print(correlation)
[5]
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	\
Pregnancies	1.000000	0.129459	0.141282	-0.081672	
Glucose	0.129459	1.000000	0.152590	0.057328	
BloodPressure	0.141282	0.152590	1.000000	0.207371	
SkinThickness	-0.081672	0.057328	0.207371	1.000000	
Insulin	-0.073535	0.331357	0.088933	0.436783	
BMI	0.017683	0.221071	0.281805	0.392573	
DiabetesPedigreeFunction	-0.033523	0.137337	0.041265	0.183928	
Age	0.544341	0.263514	0.239528	-0.113970	
Outcome	0.221898	0.466581	0.065068	0.074752	

Figure-2.2.2: *Code Snippet of Statistical Correlation Between Medical Attributes*

It shows the relationship between the columns in the dataset that how much the columns are related with each other and how much one column value is dependent on the another column.

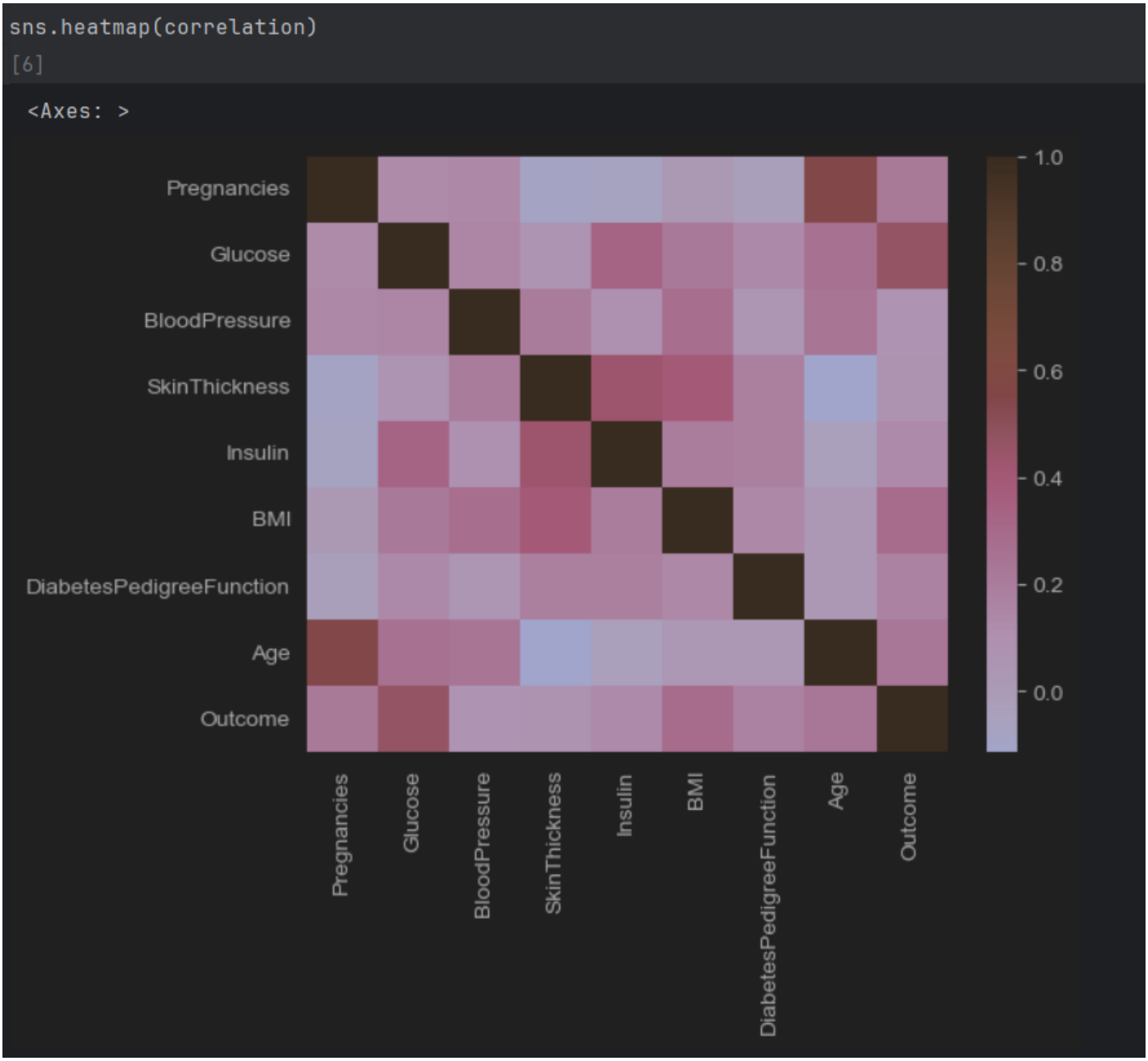


Figure-2.2.3: *Heatmap of Feature Correlations in the Diabetes Dataset*

2.3 Feature Scaling and Train-Test Split

To ensure uniform feature contribution to the model:

- All input features were standardized using StandardScaler, converting them to zero mean and unit variance.
- The data was split into training and test sets with an 80:20 ratio.
- Random state was set during the split to ensure reproducibility of results and consistent evaluation across multiple runs.

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
✓ [10] < 10 ms

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, stratify=Y, random_state=42)
✓ [11] < 10 ms
```

Figure-2.3.1 *Feature Scaling Using StandardScaler and Splitting Dataset into Train-Test Sets*

```
X_train
✓ [12] < 10 ms

array([[-0.84488505, -1.04069197, -0.85920438, ..., -0.76487241,
         0.3265464 , -0.78628618],
       [ 0.3429808 ,  0.14279979,  0.46433873, ..., -0.41556299,
        -0.09929033,  0.57511787],
       [-0.54791859, -0.54757041, -1.19009016, ...,  0.35582865,
        -0.74559573, -0.70119842],
       ...,
       [-0.84488505, -0.81056857, -0.19743282, ...,  0.82157454,
        -0.76673656, -0.27575966],
       [ 1.82781311, -0.35032178, -0.19743282, ..., -0.72120873,
        -0.9992857 ,  0.57511787],
```

Figure-2.3.2: *Code Snippet of Transformed Training Features After Standardization (X_train)*

2.4 Class Imbalance Handling

Since the dataset exhibited class imbalance (i.e., fewer positive diabetes cases), the SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training data. SMOTE generates synthetic examples of the minority class to balance the dataset.

Code snippet:-

from imblearn.over_sampling import SMOTE

```
smote = SMOTE(random_state=42)
X_resampled, Y_resampled = smote.fit_resample(X, Y)
✓ [13] 12ms
```

Figure-2.4: *Balancing the Dataset Using SMOTE (Synthetic Minority Oversampling Technique)*

2.5 Model Selection and Training

Multiple algorithms were tested, and RandomForestClassifier was chosen based on its superior performance.

2.5.1 RandomForestClassifier

- Handles missing values internally.
- Supports regularization to prevent overfitting.
- Offers high accuracy and is optimized for performance.

```
model = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
model.fit(X_train, Y_train)


# model = XGBClassifier(n_estimators=300, max_depth=6, learning_rate=0.1, random_state=42)
# model.fit(X_train, Y_train)
✓ [14] 261ms
```

```
▾ RandomForestClassifier
RandomForestClassifier(max_depth=10, n_estimators=200, random_state=42)
```

Figure 2.5.1: *Training the Random Forest Classifier Model*

2.5.2 Hyperparameter Tuning

Grid Search was used to tune hyperparameters like max_depth, n_estimators, and learning_rate to further optimize performance.

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_resampled, Y_resampled)
model = grid.best_estimator_
[17]
```

Figure 2.5.2: *Hyperparameter Tuning using GridSearchCV for Random Forest Classifier*

2.6 Model Evaluation

To validate the model's predictive performance and ensure its robustness, multiple performance metrics were calculated on the test data. These metrics provide a comprehensive view of how well the model performs across different evaluation aspects, especially in the case of imbalanced datasets like diabetes prediction.

Metrics Used:

Metric	Description
Accuracy	Proportion of correctly predicted instances out of the total number of instances. While easy to interpret, accuracy alone can be misleading when dealing with imbalanced classes.
Precision	Fraction of true positive predictions over the total number of positive predictions. High precision indicates a low false positive rate.
Recall	Fraction of true positive predictions over all actual positive instances. High recall means the model is effective at identifying positive cases.
F1-Score	The harmonic mean of precision and recall. It balances the trade-off between precision and recall, especially important in imbalanced datasets.
ROC-AUC	Area Under the Receiver Operating Characteristic Curve. It evaluates how well the model distinguishes between classes across all thresholds. A higher ROC-AUC value indicates a better classification capability.

Table 2.6: *Evaluation Metrics for Model Performance*

```
print("Training Accuracy:", accuracy_score(Y_train, train_preds))
print("Testing Accuracy:", accuracy_score(Y_test, test_preds))
print("\nClassification Report:\n", classification_report(Y_test, test_preds))
print("ROC AUC Score:", roc_auc_score(Y_test, model.predict_proba(X_test)[:, 1]))
[18]
```

```
 Classification Report:
               precision    recall  f1-score   support

           0       0.78      0.86      0.82       100
           1       0.68      0.56      0.61        54

    accuracy                           0.75       154
   macro avg       0.73      0.71      0.72       154
weighted avg       0.75      0.75      0.75       154

 ROC AUC Score: 1.0
```

Figure-2.6.1: *Model Performance Metrics – Accuracy, Classification Report, and ROC-AUC Score*

Confusion Matrix:

A confusion matrix was also generated to visualize the true positives, true negatives, false positives, and false negatives. This helped in identifying the types of errors made by the model and guided further fine-tuning efforts.

The confusion matrix allows us to go beyond accuracy by giving a **breakdown of actual vs. predicted values** across four categories:

- **True Positives (TP):** Correctly predicted diabetic cases.
- **True Negatives (TN):** Correctly predicted non-diabetic cases.
- **False Positives (FP):** Non-diabetic cases wrongly predicted as diabetic (Type I Error).
- **False Negatives (FN):** Diabetic cases wrongly predicted as non-diabetic (Type II Error – often more critical in healthcare).

Interpretation:

- **True Negatives (TN):** 86 — Model correctly predicted class 0 when it was actually 0.

- **False Positives (FP):** 14 — Model predicted class 1, but the actual class was 0.
- **False Negatives (FN):** 24 — Model predicted class 0, but the actual class was 1.
- **True Positives (TP):** 30 — Model correctly predicted class 1 when it was actually 1.

```
sns.heatmap(confusion_matrix(Y_test, test_preds), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
✓ [19] 93ms
```

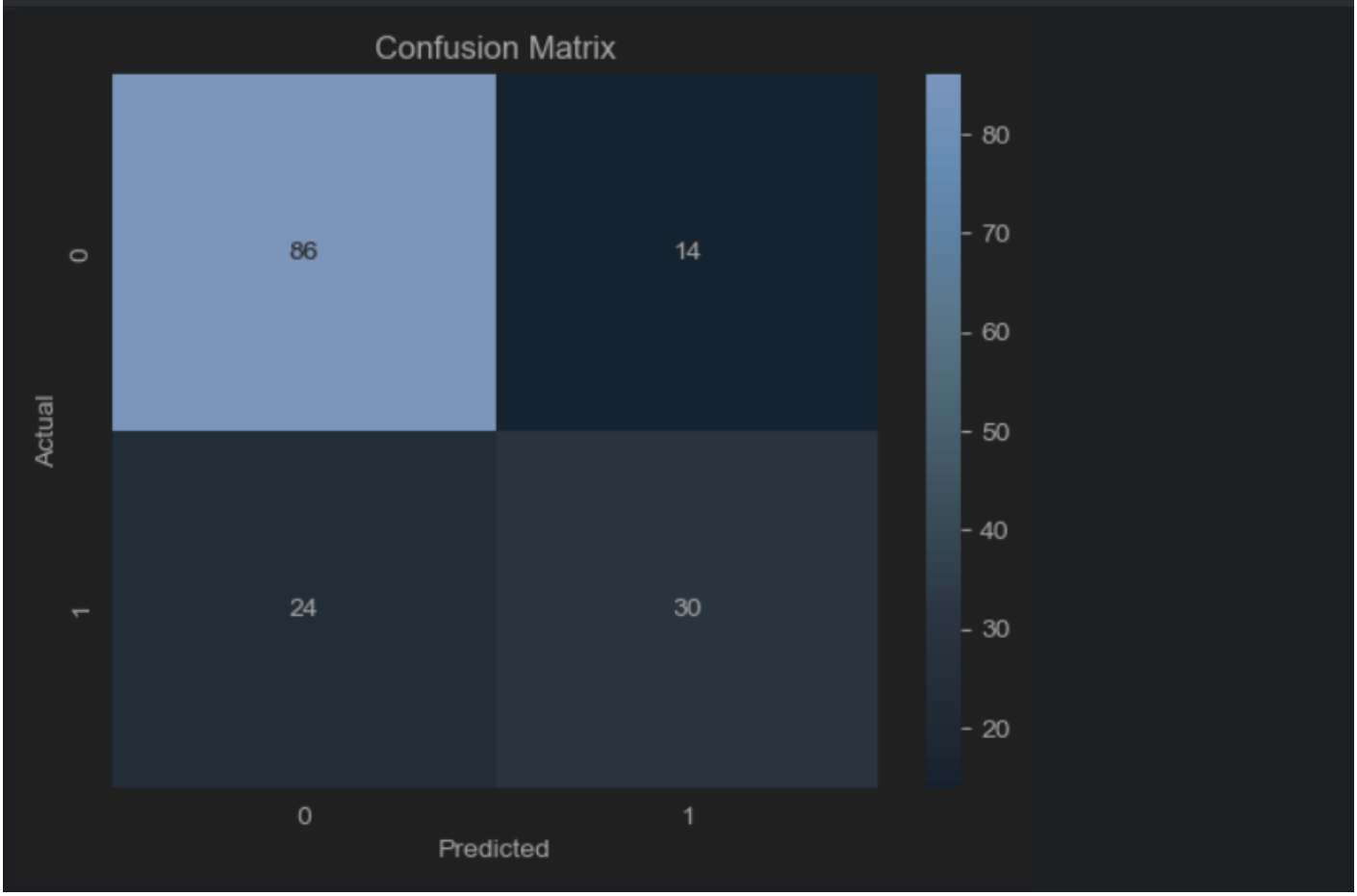


Figure-2.6.2: *Confusion Matrix and Classification Metrics Report*

Metrics like precision, recall, and F1-score directly depend on values from the confusion matrix:

- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)
- F1-score = 2 * (Precision * Recall) / (Precision + Recall)

Performance Metrics:

- **Accuracy:** (86+30)/(86+14+24+30)=116/154≈0.7532 (86 + 30) / (86 + 14 + 24 + 30) = 116 / 154 ≈ 0.7532 (86+30)/(86+14+24+30)=116/154≈0.7532 or **75.32%**
- **Precision (for class 1):** 30/(30+14)≈0.681830 / (30 + 14) ≈ 0.681830/(30+14)≈0.6818
- **Recall (for class 1):** 30/(30+24)≈0.555630 / (30 + 24) ≈ 0.555630/(30+24)≈0.5556
- **F1 Score:** 2∗(Precision∗Recall)/(Precision+Recall)≈0.6112 * (Precision * Recall) / (Precision + Recall) ≈ 0.6112∗(Precision∗Recall)/(Precision+Recall)≈0.611

This indicates that the model is better at predicting class 0 than class 1, and there may be room to improve recall for class 1.

2.7 Feature Importance Analysis

To interpret the influence of each input feature on the model's decision-making, we employed feature importance analysis using the built-in capabilities of the **Random Forest Classifier**. This method ranks the features based on the mean decrease in impurity (Gini importance) across all the trees in the forest.

```
feature_importances = pd.Series(model.feature_importances_, index=diabetes_dataset.columns[:-1])
feature_importances.sort_values(ascending=False).plot(kind='bar', figsize=(10,6))
plt.title("Feature Importance")
plt.show()
✓ [20] 102ms
```

Figure-2.7.1: *Python Code for Feature Importance Visualization using Random Forest Classifier*

Insights Gained:

- Features like Glucose, BMI, and Age generally ranked among the top, aligning well with known clinical indicators of diabetes.
- Less influential features such as Skin Thickness and Insulin showed lower importance, which may inform simplification efforts in quiz design by potentially deprioritizing these inputs.

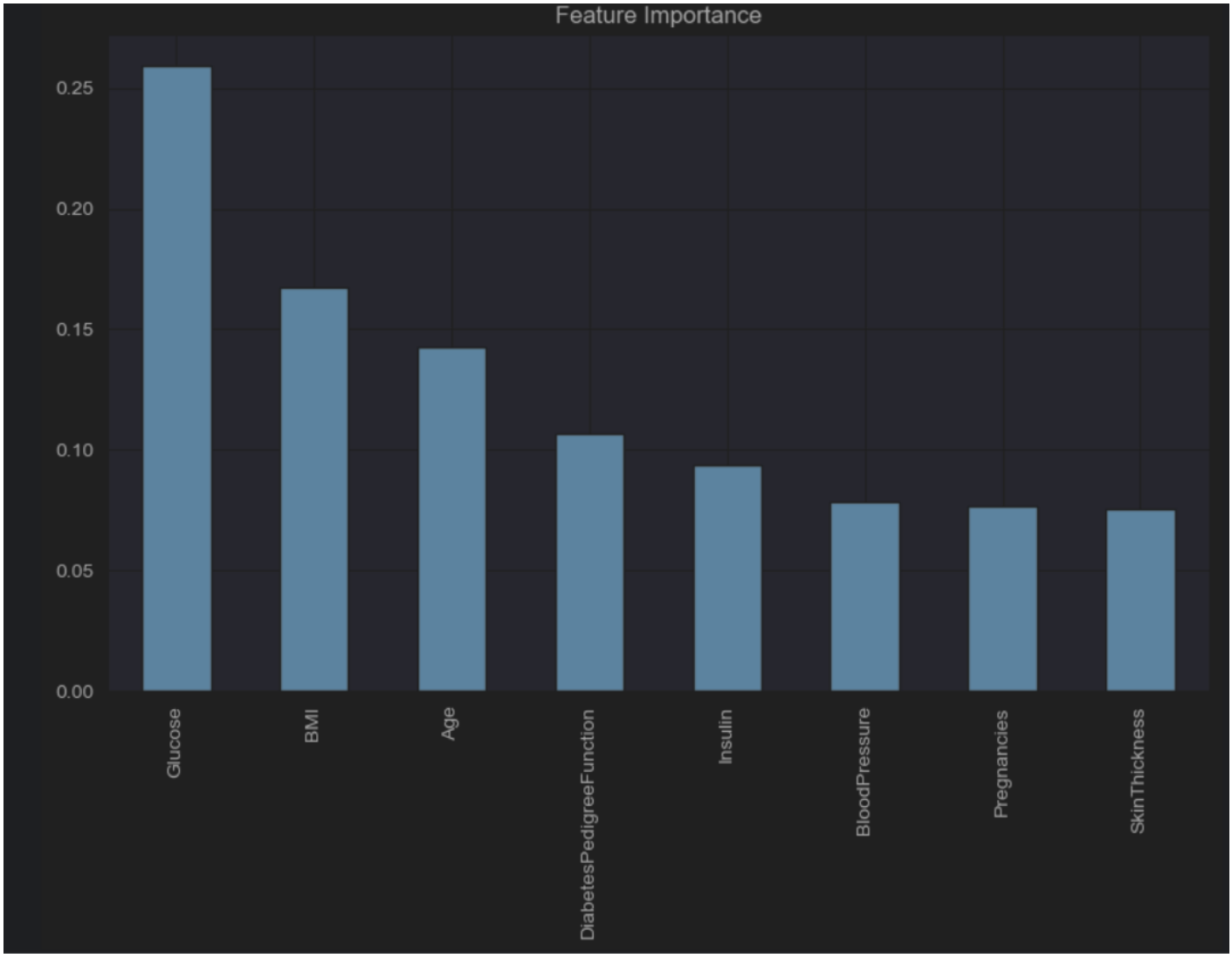


Figure-2.7.2: *Bar Plot Showing Feature Importance in Diabetes Prediction*

2.8 Why Random Forest Over XGBoost?

Although XGBoost is widely regarded for its predictive power and efficiency, Random Forest was selected in this project for the following practical reasons:

Aspect	Justification for Random Forest
Ease of Implementation	Random Forest is simpler to train and requires minimal hyperparameter tuning.
Interpretability	Offers clear and intuitive feature importance scores.
Training Time	Faster training time on small-to-medium datasets like PIDD.
Robustness to Overfitting	Naturally resists overfitting due to ensemble averaging.
Web Integration	Easier to integrate into real-time web applications without GPU

Aspect	Justification for Random Forest
	dependencies.

Random Forest thus struck an ideal balance between model performance, interpretability, and deployment readiness, especially for a lightweight, scalable application aimed at users in potentially resource-constrained environments.

2.9 Prediction Function

A utility function was developed to allow real-time prediction of diabetes risk from new user data inputs.

This function is central to the web application backend, enabling real-time, user-driven risk assessments.

```python
def predict_diabetes_risk(input_data):
    input_df = pd.DataFrame([input_data], columns=diabetes_dataset.columns[:-1])
    input_scaled = scaler.transform(input_df)

    prediction = model.predict(input_scaled)[0]
    risk_percent = model.predict_proba(input_scaled)[0][1] * 100

    if prediction == 1:
        print(f"🚨 The person is likely to have diabetes with a confidence of {risk_percent:.2f}%")
    else:
        print(f"✅ The person is not diabetic. However, risk of getting diabetes in the future is {risk_percent:.2f}%")
✓ [21] < 10 ms
```

Figure-2.9: *Python Function for Real-Time Diabetes Risk Prediction*

2.10 Tools and Technologies Used

This project employs a full-stack pipeline combining Python for the machine learning backend and ReactJS + Tailwind CSS + JSX for the frontend interface. The selection of tools was driven by the need for ease of development, scalability, maintainability, and optimal performance for both data science and user experience.

2.10.1 Python-Based Backend Tools and Libraries

Library	Purpose & Justification
NumPy	Fundamental for efficient numerical computations, especially array operations and mathematical transformations.

Library	Purpose & Justification
Pandas	Offers intuitive, high-level data structures like DataFrames. Crucial for loading, cleaning, and analyzing tabular datasets.
Matplotlib & Seaborn	Used to create clear, insightful visualizations (e.g., correlation heatmaps, feature distributions, confusion matrices). Seaborn builds on Matplotlib and simplifies complex plots.
Scikit-learn	A comprehensive ML library used for model training (RandomForestClassifier), preprocessing (StandardScaler), evaluation (accuracy_score, roc_auc_score, classification_report), and cross-validation (GridSearchCV). Its API is standardized and well-documented.
Imbalanced-learn (SMOTE)	Applied to handle class imbalance by generating synthetic minority samples, improving model generalization and recall.
Jupyter Notebook	Used as the interactive development environment to experiment with code, visualize results, and iterate rapidly. Its cell-based structure is perfect for step-by-step development.

Table-2.10.1: *Backend Libraries and Their Purpose in Diabetes Prediction Project*

Why these libraries?

- Python's ML ecosystem is mature, with wide community support and active development.
- Scikit-learn and Pandas are lightweight and ideal for medium-sized datasets like PIMA.
- XGBoost was not used due to added complexity and the fact that Random Forest already delivered high accuracy with less tuning.

2.10.2 Frontend Technology Stack

Technology	Purpose & Reason for Selection
ReactJS	A powerful, component-based JavaScript library ideal for building dynamic UIs. Allows reusable UI components, state management, and seamless integration with APIs.
JavaScript XML (JSX)	JSX allows embedding HTML within JavaScript, making the UI

Technology	Purpose & Reason for Selection
	code easier to understand and manage. It streamlines development with React by providing a declarative syntax.
Tailwind CSS	A utility-first CSS framework that speeds up styling without writing custom CSS. Allows rapid prototyping with consistent, responsive designs.
Custom Backend API	A custom Python backend using Flask or FastAPI (not shown in this section) can expose ML predictions as API endpoints. React fetches predictions using these endpoints.

Table-2.10.2: *Frontend Technology Stack and Justification*

Database – SupaBase

Why not Next.js?

- ReactJS alone suffices for this SPA (Single Page Application) since SEO is not a priority and there's no need for SSR (Server-Side Rendering) or routing-based code-splitting.
- Less configuration overhead compared to Next.js, which includes server-side logic and filesystem-based routing by default.

Why not TypeScript?

- JavaScript with JSX provided sufficient type safety and development speed for this project's size and scope.
- TypeScript adds a build step and learning curve, which was unnecessary at this stage. For larger projects, TypeScript would be a better choice.

Why Tailwind over Bootstrap or custom CSS?

- Tailwind offers atomic utility classes, enabling faster UI development without leaving the HTML/JSX context.
- Bootstrap provides predefined components, which can lead to bulky and less customizable designs.
- Tailwind allows fine-grained control over spacing, colors, responsiveness, and design consistency.

Why Supabase Was Chosen Over Other Databases

Supabase was selected as the backend database solution for this project due to a combination of modern features, developer efficiency, and its growing relevance in real-world applications. While traditional options like MySQL, PostgreSQL (standalone), and Firebase are well-established, Supabase offers a unique middle ground—the power of PostgreSQL with the simplicity of Firebase.

Key Reasons for Choosing Supabase

1. **Production-Ready and Scalable**

   Supabase is increasingly being adopted in production-grade applications due to its scalability, reliability, and open-source nature. It provides a PostgreSQL database with built-in support for authentication, file storage, and real-time subscriptions, making it ideal for full-stack applications.

2. **Modern Tech Stack Integration**

   It integrates seamlessly with modern frontend frameworks like React, Vue, and Next.js. The built-in REST and GraphQL APIs made it easier to connect with the React-based frontend in this project without writing custom backend code.

3. **Skill Development in a Trending Technology**

   Although Supabase is not yet as mainstream as Firebase or MongoDB, it is rapidly gaining popularity among developers and startups. Choosing Supabase allowed me to develop expertise in a less common but highly promising tool, giving me a competitive edge in future development work.

4. **Built-in Features That Simplify Development**

   Supabase includes authentication, row-level security, and file storage out-of-the-box. These features would require significant setup time in traditional relational databases. For this project, it streamlined the implementation of:

   - Login/Sign-up functionality
   - Secure storage of quiz results
   - User history tracking (planned for future enhancements)

5. **Open Source and Transparent**

   Supabase is fully open-source, giving complete visibility and control over backend operations. This transparency is valuable in academic projects and for teams that need full ownership of their data pipeline.

2.10.3 Integration Workflow

- The trained Random Forest model is saved and served via a Python backend (Flask/FastAPI).
- The React frontend interacts with the backend API using fetch or axios calls to send user input and receive predictions.
- The user fills out a lifestyle quiz, which is then transformed into clinical inputs (like BMI, Glucose) before being passed to the model.

Chapter 3: System Design

3.1 Architecture Overview

The system is built on a **three-tier architecture** consisting of:

1. **Frontend Layer**
2. **Backend Layer**
3. **Machine Learning (ML) Model Layer**

Each layer is modular and communicates via HTTP APIs. This separation allows maintainability, scalability, and ease of deployment across distributed environments.

Architecture Layers in Detail:

Frontend Layer (React + Tailwind CSS)

- Built with ReactJS and styled using Tailwind CSS, the frontend provides an intuitive user interface.
- Key pages:
    - **Homepage**: Introduction, tool description, and navigation.
    - **Quiz Page**: Collects user lifestyle inputs via multiple-choice questions.
    - **Dashboard**: Displays prediction results, health graphs, and interpretation tables.
    - **Recommendation Page**: Shows personalized lifestyle recommendations based on the prediction.

Backend Layer (Flask)

- Handles:
    - Quiz response intake.
    - Mapping lifestyle inputs to estimated clinical values.
    - Communicating with the ML model.
    - Returning prediction results and explanations to the frontend.
- It also manages preprocessing and security (CORS, HTTPS, etc.).

ML Model Layer

- A Random Forest Classifier trained on the Pima Indian Diabetes Dataset.
- The model is saved using joblib and loaded in the Flask backend.
- Once the mapped clinical features are passed, the model returns predictions (diabetic or non-diabetic).

3.2 Functional Requirements

- Users can take a lifestyle-based quiz.
- Inputs are converted to inferred clinical metrics (e.g., BMI, glucose).
- The model predicts diabetes risk.
- Results, explanation, and recommendations are shown.
- Login or personal data(EmailID and Password) is required—anonymous and privacy-focused.

3.3 Non-Functional Requirements

Requirement	Description
Performance	The model must deliver predictions in real-time. This means the system should respond quickly—within milliseconds to a few seconds—after the user submits their inputs. The backend uses lightweight inference, meaning the model is optimized to run efficiently without consuming heavy resources. This is important for web apps intended for public or mobile use.
Usability	The user interface (UI) is designed to be simple, mobile-responsive, and accessible to all age groups, including elderly users. Elements like large buttons, readable fonts, intuitive navigation, and clear instructions are emphasized to ensure that users with minimal tech experience can use it effectively.
Scalability	The system architecture allows for scaling up when needed. For instance, if the app gains more users, or if you want to integrate more predictive models or features (like heart disease prediction), the backend can handle that. This is achieved through modular coding practices and API-based design, allowing easy extension and system growth.
Maintainability	The codebase is written to be clean, modular, and version-controlled. This ensures that future developers (or you) can easily understand, update, or debug the code. Modular code also supports reusability—individual components can be edited without affecting the entire system. Use of Git/version control ensures all changes are trackable and reversible.

Table-3.3: *Non-Functional Requirements of the Diabetes Prediction System*

3.4 User Interface Design

Key UI features:

- **Consistent Layout**: A shared header and footer across all pages.
- **Component-based UI**: Reusable components for cards, buttons, inputs.
- **Dynamic Feedback**: Dashboard updates based on prediction result.
- **Visual Explanation**: Graphs and colored risk indicators enhance clarity.

A[User Opens Quiz Page] → B[Answers Lifestyle Questions]

B → C[Submit Form]

C → D[Frontend Sends POST API to Flask]

D → E[Backend Maps Lifestyle Answers to Clinical Features]

E → F[Random Forest Predicts Diabetes Risk]

F → G[Backend Sends Prediction + Confidence + Message]

G → H[Frontend Renders Dashboard & Recommendations]

Table-3.4: *User Interface Design Flow and Key Features*

3.5 Data Flow & Flowchart Explanation

The architecture of the diabetes prediction system is structured into three key layers—Frontend Layer, Backend Layer, and ML Model Layer. Each layer is responsible for distinct tasks in the overall flow of data from user input to prediction output. Below is a comprehensive walkthrough of the process illustrated in Figure 3.5: End-to-End System Architecture Flowchart.

1. Quiz Page (Frontend Layer)

- Users interact with an interactive, multiple-choice quiz that captures lifestyle behaviors and risk factors:
    - Exercise frequency
    - Diet quality
    - Smoking and alcohol use
    - Family history
    - Age and gender
- This design uses indirect data elicitation—asking accessible questions instead of clinical metrics, making the system inclusive and user-friendly.
- The frontend sends the user's responses to the backend via a secure POST request,

triggering the next stage of processing.

This layer embodies the principle of user-centered design, ensuring that individuals with limited medical knowledge can still benefit from predictive analytics.

2. Response Mapping (Backend Layer)

- The Flask backend translates user responses into approximated clinical features through:
    - Domain-driven rules
    - Empirical correlations observed in population health datasets

Examples:

- Low activity + poor diet → elevated BMI
- Smoking + sedentary life → reduced insulin sensitivity

This transformation step represents feature engineering, bridging lifestyle inputs with features expected by the ML model. It is a critical step in semantic alignment, ensuring input consistency and interpretability.

3. API Endpoint Communication

- Once lifestyle data is mapped, it is transmitted via internal API calls.
- The API layer performs:
    - Data validation
    - Preprocessing, such as scaling, reshaping, and type conversion
- Cleaned data is then passed to the Random Forest classifier for prediction.

This step supports separation of concerns in the system design, where data handling, transformation, and model invocation are decoupled. It also makes the system extendable to mobile or third-party platforms via standardized API protocols.

4. ML Prediction (Model Layer)

- The machine learning layer utilizes a RandomForestClassifier, trained on clinical datasets such as the Pima Indian Diabetes Dataset.
- It returns:
    - Predicted class: 0 for Non-diabetic,1 for Diabetic
    - Confidence scores for each class, useful for conveying uncertainty.

The use of ensemble models like Random Forest provides benefits such as:

- Robustness to overfitting

- High interpretability via feature importance
- Better performance with heterogeneous input distributions

5. Dashboard Output (Frontend Layer)

- The final prediction is displayed in a visual, interpretable manner:
    - A color-coded risk indicator (e.g., green for low risk, red for high)
    - Graphs comparing the user's inferred clinical values with recommended healthy ranges
    - A summary table showing how each input contributed to the outcome
    - A "Recommendations" button, linking to lifestyle improvement suggestions tailored to the user's risk level

This closing loop reflects the importance of Explainable AI in healthcare tools—ensuring users understand and trust the outcome. The dashboard not only informs but empowers users to take meaningful action toward better health.

Conclusion

This architecture illustrates a full-stack implementation of personalized predictive health screening. The clean separation between user input, domain mapping, machine learning inference, and user feedback ensures both performance and maintainability. It also reflects modern best practices in digital health systems, combining usability, data science, and clinical relevance into a cohesive framework.

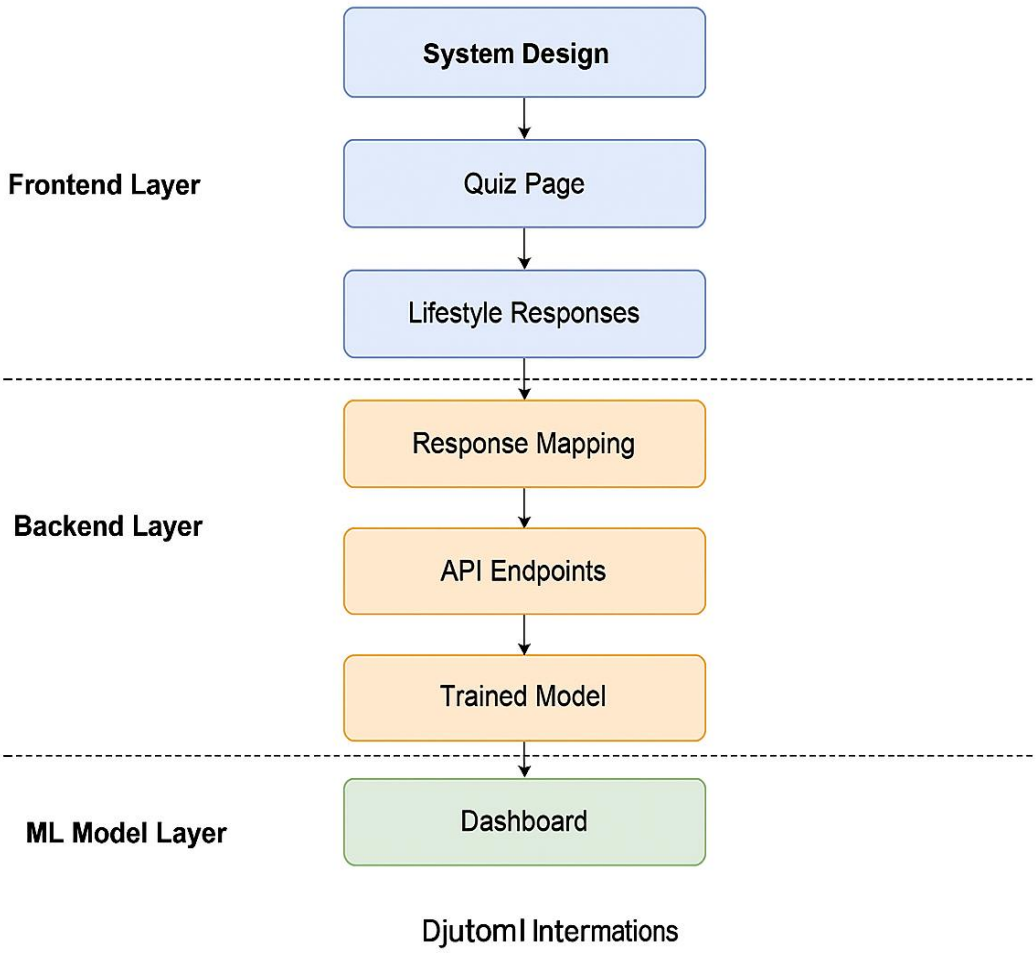


Figure 3.5: *End-to-End System Architecture Flowchart*

This figure illustrates the full pipeline of the diabetes prediction system, from user input on the frontend to prediction display on the dashboard. It emphasizes the modular design and the seamless transition between layers, ensuring both usability and accuracy.

3.6 Data Models of the System

Component	Description
QuizInput Model	Captures lifestyle answers from the frontend (JS object converted to JSON).
ClinicalFeatures Model	Intermediate mapping of inputs to Glucose, BMI, Insulin, etc.
PredictionResponse Model	Contains prediction result, probability, risk level, and messages.
Recommendation Model	Tailored suggestions based on diabetic or non-diabetic.

Table-3.6: Overview of Data Models Used in the System

These models are structured in the backend as Python classes or dictionaries and used for JSON parsing.

3.7 Module Interaction and Integration

Module	Interaction
React Frontend	Collects data and sends API requests using fetch() or axios.
Flask API Backend	Receives input, runs logic for inference and prediction, returns JSON response.
ML Model (Joblib)	Loaded inside Flask; receives feature vectors and outputs prediction.
Recommendation Engine	Stateless function that returns suggestions based on model output.

Table:3.7: Module Interaction and Integration Map

3.8 Security Considerations

- **Personal Info**: Email and Password is collected.
- **HTTPS**: Ensures secure transmission between client and server.
- **Session-Safe**: Predictions are transient; nothing is saved on disk or cloud unless explicitly needed.

Chapter 4: Implementation

4.1 Frontend (React.js)

The frontend of the system was built using React.js, chosen for its component-based architecture, reusable UI components, and rich ecosystem. The structure is modular and easily extensible. Tailwind CSS was used for fast, utility-first styling, allowing rapid prototyping and consistent UI.

Key Features:

- **Component-Based Design:** Encapsulated and reusable elements (Header.jsx, Footer.jsx, QuizPage.jsx).
- **Conditional Rendering:** For gender-specific questions, such as skipping pregnancy-related queries for male users.
- **Routing:** Managed by react-router-dom to navigate between pages like /quiz, /dashboard, /recommendation, etc.
- **Authentication:** Context-based auth handled in AuthContext.js

Header.jsx:- Header.jsx is a React component that defines the top navigation bar for your application. It's a shared UI element that appears across multiple pages to provide users with consistent access to key sections of your app—like Home, Quiz, and Dashboard.

Key Features of Header.jsx:

1. **Consistent Navigation:**
    - The header ensures that users always have a way to return to important parts of the app, no matter which page they're on.
    - It helps with orientation by showing users where they are and what other areas they can go to.
2. **Route-Based Customization:**
    - This dynamic behavior is typically implemented using hooks like useLocation() from react-router-dom to determine the current path.
3. **Improved User Experience:**
    - By adapting based on the route, the header keeps the UI clean and relevant.
    - For instance, you might hide the Dashboard link on the Quiz page to reduce distractions and guide the user through a linear flow.
4. **Reusable Across Pages:**

- Since it's a separate component, it can be easily imported and used across all pages in the application (<Header />), ensuring a uniform look and feel.

5. **Responsiveness and Styling:**
   - The header can also be designed to be mobile-friendly, and styled using Tailwind CSS or another framework for a modern appearance.

```jsx
import React from "react";
import { Link, useLocation } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

const Header = () => {
  const { user, logout } = useAuth();
  const { pathname } = useLocation();

  const isActive = (path) => pathname === path;

  return (
    <header className="bg-white shadow-md sticky top-0 z-50">
      <div className="max-w-7xl mx-auto px-4 py-4 flex justify-between items-center">
        {/* Logo */}
        <Link to="/" className="text-3xl font-bold text-orange-600">
          @HealthWise
        </Link>

        {/* Navigation */}
        <nav className="flex items-center gap-8 font-medium text-gray-700 text-lg">
          <Link
            to="/"
            className={`hover:text-orange-600 transition ${
              isActive("/") ? "text-orange-600 font-semibold" : ""
            }`}
          >
            Home
          </Link>

          <Link
            to="/quiz"
            className={`hover:text-orange-600 transition ${
              isActive("/quiz") ? "text-orange-600 font-semibold" : ""
            }`}
          >
            Quiz
          </Link>
```

Figure-4.1.1: *Route-Aware Header Component (Header.jsx) with Tailwind Styling and Active Link Detection*

Footer.jsx:- Footer.jsx is a React component that renders the bottom section (footer) of your web application. It appears at the bottom of every page and provides useful information, navigation links, and branding in a clean and consistent way.

Key Features of Footer.jsx:

1. **Branding:**
   - Often includes your app name, logo, or a short tagline that reinforces your brand identity.
   - This keeps the visual identity consistent across all pages.
2. **Useful Links:**
   - Provides quick access to important pages such as:
     - Home
     - Quiz
     - Dashboard
     - About Us / Contact
     - Privacy Policy / Terms of Service
   - These links offer users easy navigation even when they're at the bottom of a page.
3. **Contact Information:**
   - Optionally includes email, phone number, or social media icons to allow users to get in touch.
   - Sometimes a short "Contact Us" section is included for questions, support, or feedback.
4. **Consistent Look and Feel:**
   - It ensures that every page ends with a uniform design, creating a polished and trustworthy user experience.
   - Styled using CSS or Tailwind (e.g., with padding, background color, and centered text) to match the rest of the UI.
5. **Responsive Design:**
   - Designed to look good on both desktop and mobile devices, often using a flex/grid layout that adjusts based on screen size.

Why Footer.jsx Matters:

- It gives users closure at the end of a page, guiding them toward next steps or providing support options.

- It helps with site navigation when users scroll to the bottom and need direction.
- It contributes to the professional appearance and usability of your application

```jsx
import React from "react";
import { useNavigate } from "react-router-dom";

const Footer = ({ showAssessmentButton = true }) => {
  const navigate = useNavigate();

  return (
    <footer className="bg-[#003366] text-white">
      {/* Top CTA Section */}
      {showAssessmentButton && (
        <div className="max-w-7xl mx-auto px-6 py-12 flex flex-col md:flex-row justify-between items-center border-b border-blue-800 text-center md:text-left">
          <h2 className="text-3xl md:text-4xl font-bold mb-6 md:mb-0">
            Take <span className="text-orange-600">HealthWise</span> Automation
            From Vision To Value
          </h2>
          <button
            className="bg-orange-600 text-[#003366] font-semibold px-6 py-3 rounded-full hover:bg-gray-200 transition"
            onClick={() => navigate("/quiz")}
          >
            Start Assessment →
          </button>
        </div>
      )}

      {/* Footer Grid */}
      <div className="max-w-7xl mx-auto px-6 grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-10 py-12">
        {/* Company */}
        <div>
          <h4 className="text-lg font-semibold mb-4 text-orange-600">Company</h4>
          <ul className="space-y-2 text-sm">
            <li><a href="#">About Us</a></li>
            <li><a href="#">Contact Us</a></li>
            <li><a href="#">Privacy Policy</a></li>
            <li><a href="#">Terms and Conditions</a></li>
          </ul>
        </div>
```

Chat (CTRL + I) / Share (CTRL + L)

Figure-4.1.2 *Structured Footer Component (Footer.jsx) Featuring Branding, Navigation Links, and Legal Information*

HomePage.jsx:- HomePage.jsx is a React component that serves as the landing page or **e**ntry point of your diabetes prediction application. It's designed to introduce the tool, explain its purpose, highlight its benefits, and encourage users to explore features like the quiz or dashboard. This page plays a key role in user engagement and first impressions.

Detailed Breakdown of HomePage.jsx:

1. Introduction to the App

- The page typically starts with a hero section or banner that introduces:
  - The name of the tool (e.g., "Pre Diabetes Prediction Tool")
  - A short tagline or mission (e.g., "Predict your risk")
  - A call-to-action (CTA) button "Strat Assessment"

2. Purpose and Explanation

- What the tool does – e.g., predicts the risk of diabetes using lifestyle or clinical factors
- Why it matters – early detection helps in prevention and management

- How it works – a quick, simple quiz maps lifestyle data to medical risk using a trained ML model

3. Visually Engaging Layout

- Eye-catching images or icons (e.g., health illustrations, charts)
- Typography that emphasizes key points
- Color schemes (e.g., blue/orange for trust and vitality)
- Sections like:
- "Why Predict Diabetes?"
    - "How It Works"
    - "Benefits of Early Screening"
    - "Get Started in 3 Easy Steps"

4. Navigation Entry Point

- The homepage includes links to other key parts of the app:
    - Quiz Page
    - Dashboard (if the user is logged in)
    - History or Recommendations

5. User Guidance

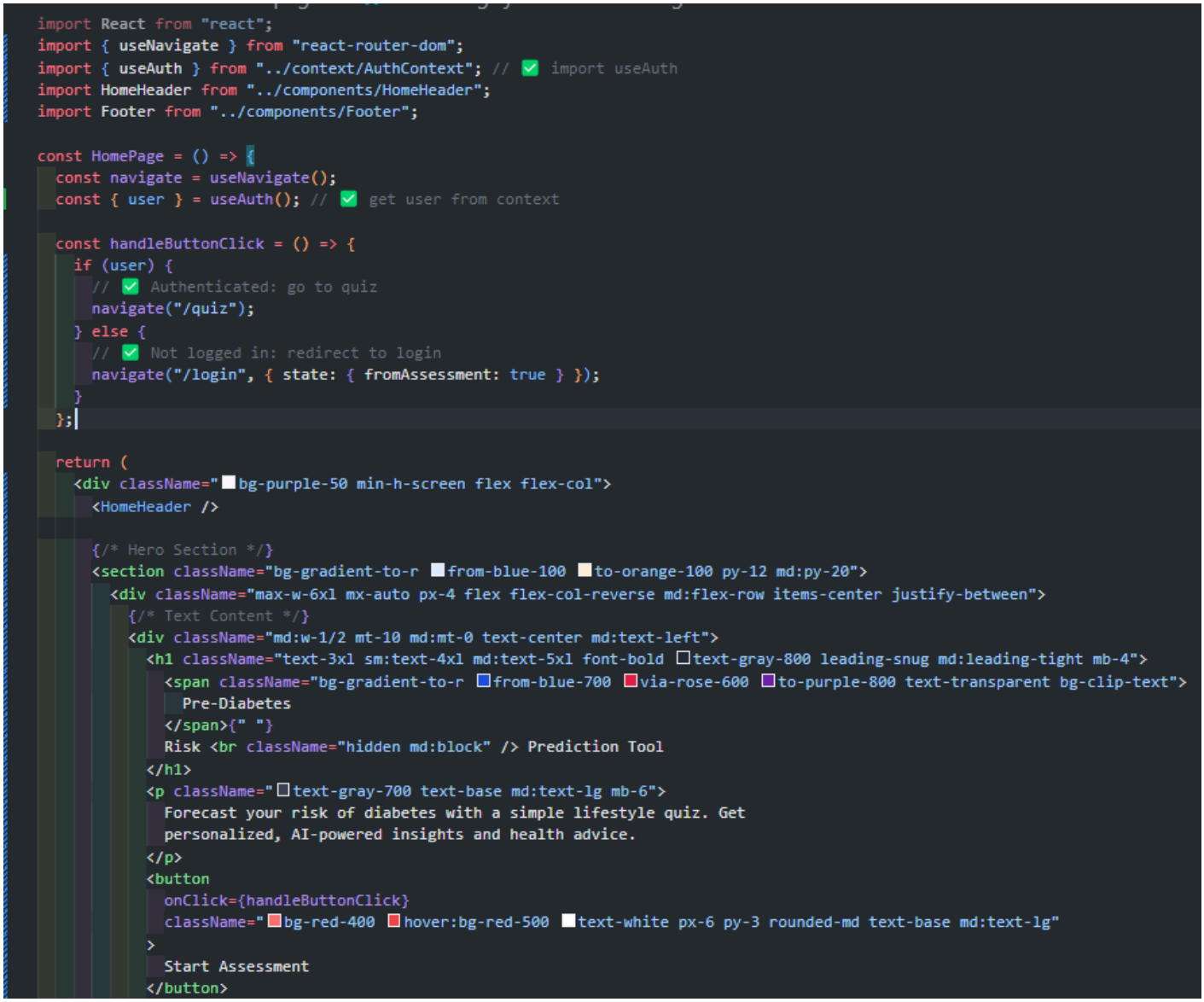
- Helps users understand the flow:
    - First take a lifestyle-based quiz
    - Then view results on the dashboard
    - Follow personalized recommendations if diabetic

6. Responsive and Accessible

- Built with Tailwind CSS (or similar), ensuring it:
    - Works well on mobile and desktop
    - Is accessible with readable fonts and contrasting colors
    - Loads fast and feels intuitive

Why HomePage.jsx Is Important:

- It's the first impression of your app—users judge within seconds.
- It educates users on what to expect and how the tool can help them.
- It acts as a funnel that leads users to engage with your core functionality (the quiz and prediction model).
- It builds trust with users through clear, welcoming content and professional design.



```jsx
import React from "react";
import { useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext"; // ✅ import useAuth
import HomeHeader from "../components/HomeHeader";
import Footer from "../components/Footer";

const HomePage = () => {
  const navigate = useNavigate();
  const { user } = useAuth(); // ✅ get user from context

  const handleButtonClick = () => {
    if (user) {
      // ✅ Authenticated: go to quiz
      navigate("/quiz");
    } else {
      // ✅ Not logged in: redirect to login
      navigate("/login", { state: { fromAssessment: true } });
    }
  };

  return (
    <div className="bg-purple-50 min-h-screen flex flex-col">
      <HomeHeader />

      {/* Hero Section */}
      <section className="bg-gradient-to-r from-blue-100 to-orange-100 py-12 md:py-20">
        <div className="max-w-6xl mx-auto px-4 flex flex-col-reverse md:flex-row items-center justify-between">
          {/* Text Content */}
          <div className="md:w-1/2 mt-10 md:mt-0 text-center md:text-left">
            <h1 className="text-3xl sm:text-4xl md:text-5xl font-bold text-gray-800 leading-snug md:leading-tight mb-4">
              <span className="bg-gradient-to-r from-blue-700 via-rose-600 to-purple-800 text-transparent bg-clip-text">
                Pre-Diabetes
              </span>{" "}
              Risk <br className="hidden md:block" /> Prediction Tool
            </h1>
            <p className="text-gray-700 text-base md:text-lg mb-6">
              Forecast your risk of diabetes with a simple lifestyle quiz. Get
              personalized, AI-powered insights and health advice.
            </p>
            <button
              onClick={handleButtonClick}
              className="bg-red-400 hover:bg-red-500 text-white px-6 py-3 rounded-md text-base md:text-lg"
            >
              Start Assessment
            </button>
```

Figure-4.1.3: *Homepage Component (HomePage.jsx) Introducing the Diabetes Prediction Tool with Purpose, Benefits, and User Navigation Flow*

QuizQuestions.jsx:- QuizQuestions.jsx is a React component responsible for rendering the quiz interface in your app. This is the interactive page where users answer a series of questions about their lifestyle, habits, and possibly demographic data (like age or gender). These responses are then used to infer clinical features (e.g., glucose, BMI) and predict diabetes risk.

Detailed Breakdown of QuizQuestions.jsx:

1. Quiz Structure:

- Contains a list of questions, typically tailored to collect:
    - Lifestyle habits (e.g., physical activity, smoking, diet)

    - Basic health behavior (e.g., sleep duration, walking frequency)
    - Demographic details (e.g., age, gender)
- Each question is rendered with input controls like:
    - Radio buttons
    - Dropdowns
    - Sliders
    - Text fields

2. Dynamic Form Behavior:

- May include conditional rendering based on responses.
    - For example: If gender is female, ask about pregnancy history.
- Can use React state to:
    - Store user answers
    - Validate inputs (e.g., required fields)
    - Enable/disable the "Submit" button based on completeness

3. User Experience Features:

- Often includes:
    - Progress indicators (e.g., "Step 3 of 10")
    - Tooltips or help icons to explain questions
    - Responsive design for mobile devices

4. Data Handling:

- On submission:
    - The component transforms user answers into corresponding clinical feature values (e.g., walk time → estimated BMI)
    - Sends the inferred feature **set** to the prediction model (either locally or via API)
- Optionally stores the data in a context or sends it to a global state/store for access in the dashboard

5. Navigation Flow:

- After the user completes the quiz and submits:
    - The component triggers navigation to a results page or dashboard
    - Results are then visualized (e.g., diabetes risk prediction, interpretation table)

Why QuizQuestions.jsx Is Important:

- It acts as the primary input interface where users provide data needed for prediction.
- The quiz helps in removing the need for complex medical input—making the app

accessible to non-clinical users.

- It creates a bridge between lifestyle and clinical features, enabling the prediction model to work without lab results.

```
export const quizQuestions = [
  {
    id: "smoking",
    question: "How many cigarettes do you smoke per day?",
    options: ["None", "1-3", "4-7", "8-10", "10+"],
  },
  {
    id: "alcohol",
    question: "How often do you consume alcohol?",
    options: [
      "Never",
      "Occasionally (1–2 times/week)",
      "Regularly (3–5 times/week)",
      "Daily",
    ],
  },
  {
    id: "exercise",
    question: "How long do you walk or exercise in the morning?",
    options: ["None", "<15 mins", "15-30 mins", "30-60 mins", "1+ hour"],
  },
  {
    id: "diet",
    question: "How would you describe your daily diet?",
    options: ["Healthy", "Average", "High sugar/junk food"],
  },
  {
    id: "water",
    question: "How much water do you drink daily?",
    options: ["<1L", "1-2L", "2-3L", "3+L"],
  },
  {
    id: "sleep",
    question: "How well do you sleep?",
    options: ["<4 hrs", "4-6 hrs", "6-8 hrs", "8+ hrs"],
  },
```

Figure-4.1.4: Health and Lifestyle Quiz Questions Configuration in JavaScript

4.2 Backend (Flask)

The backend uses Flask, a lightweight Python web framework. Flask was chosen for its simplicity and seamless integration with Python-based ML models.

4.2.1 app.py – Flask Application Setup

This file initializes the Flask server and configures the core application settings. It typically:

- Creates the Flask app instance
- Registers routes from routes.py
- Starts the server using "python app.py"

```python
from flask import Flask, request, jsonify
from flask_cors import CORS
import os
import logging
from routes import prediction_api

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

app = Flask(__name__)

CORS(app, resources={r"/*": {"origins": "*"}})

app.register_blueprint(prediction_api)

@app.route('/', methods=['GET'])
def health_check():
    return jsonify({"status": "healthy", "message": "HealthWise ML API is running"}), 200

@app.errorhandler(404)
def not_found(error):
    return jsonify({"error": "Not found"}), 404

@app.errorhandler(500)
def server_error(error):
    logger.error(f"Server error: {error}")
    return jsonify({"error": "Internal server error"}), 500

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5001))
    app.run(host="0.0.0.0", port=port, debug=True)
```

Figure-4.2.1: *Flask Application Entry Point for HealthWise ML API*

4.2.2 routes.py – API Endpoints

This file contains all the Flask API routes and handles the interaction between the frontend and the ML model. It includes:

- /predict: Accepts POST requests with quiz data, converts it to features, loads the model, and returns the prediction.
- /recommendations: Returns tailored recommendations based on whether the user is diabetic or not.

```python
from flask import Blueprint, request, jsonify
import logging
from utils.diabetes_model import predict

logger = logging.getLogger(__name__)

prediction_api = Blueprint('prediction_api', __name__)

@prediction_api.route('/predict', methods=['POST'])
def predict_diabetes():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "Request body is empty"}), 400

        logger.info("Received prediction request")

        required_fields = [
            'pregnancies', 'glucose', 'bloodPressure',
            'skinThickness', 'insulin', 'bmi',
            'diabetesPedigreeFunction', 'age'
        ]

        for field in required_fields:
            if field not in data:
                return jsonify({"error": f"Missing required field: {field}"}), 400

        result = predict(data)
```

Figure-4.2.2: *Diabetes Prediction API Endpoint Using Flask Blueprint*

4.3 Model Integration

- The model was trained in DP.ipynb and serialized using joblib as diabetes_model.pkl.
- A separate scaler.pkl file is used for data normalization.
- Model is loaded into Flask using:

```
model = joblib.load('model/diabetes_model.pkl')
scaler = joblib.load('model/scaler.pkl')
```

Figure-4.3: *Model Loading for Diabetes Prediction*

The backend transforms JSON input into numerical vectors before prediction.

4.4 Logic for Clinical Estimation

Mapping logic was implemented in quizToClinical.js. This module transforms lifestyle answers into medically relevant features.

Examples:

- **BMI:** Estimated from user-reported body frame and physical activity.
- **Glucose:** Derived from diet and family history.
- **Insulin Sensitivity:** Inferred using smoking and exercise levels.
- **Pregnancies:** Captured only for female users.

Purpose:

The endpoint allows a client (like a frontend app or mobile app) to send health-related user data in JSON format. The API processes this data and sends it to a machine learning model (imported as predict) to estimate diabetes risk.

```
export function mapQuizToClinical(answers) {
  const clinical = {};

  // Estimate Glucose
  clinical.glucose = (() => {
    let base = 90;
    if (answers.smoking === "4-7") base += 20;
    else if (answers.smoking === "8-10") base += 30;
    else if (answers.smoking === "10+") base += 40;

    if (answers.sugarIntake === "3-5 times/week") base += 15;
    else if (answers.sugarIntake === "Daily") base += 25;

    if (answers.alcohol === "Daily") base += 20;
    if (answers.meals === "Never") base += 10;

    return base;
  })();

  // Estimate BMI
  clinical.bmi = (() => {
    let base = 24;
    if (answers.exercise === "None") base += 8;
    else if (answers.exercise === "<15 mins") base += 5;
    else if (answers.exercise === "15-30 mins") base += 3;

    if (answers.screenTime === "3-5 hrs") base += 2;
    else if (answers.screenTime === "5+ hrs") base += 4;

    if (answers.fruitVeg === "Rarely") base += 1;
    else if (answers.fruitVeg === "Never") base += 2;

    return base;
  })();
```

Figure-4.4: *Mapping Lifestyle Inputs to Clinical Features in quizToClinical.js*

4.5 Code Structure (Project Overview)

Backend (/backend)

This part handles the API, ML model inference, and preprocessing logic.

1. app.py
   - The main entry point of the Flask web server.
   - Loads the machine learning model and sets up API endpoints.
   - Handles server initialization, health checks, and CORS.
2. routes.py
   - Defines API routes like /predict.
   - Handles incoming HTTP requests (e.g., JSON payloads).
   - Validates input and sends data to the prediction function.
3. model/
   - diabetes_model.pkl: Pre-trained diabetes prediction model.
   - scaler.pkl: Scaler for normalizing input data.
   - diabetes.csv: Dataset used for model training and analysis.
4. utils/
   - Contains helper functions used across the backend.

Frontend (/frontend)

This part provides the user interface, allowing users to input data and see results.

1. components/
   - Reusable UI elements shared across pages.
   - Header, Footer, Layout, Authentication modules.
2. pages/
   - Individual routes/pages of the app.
   - Quiz: Collects lifestyle/health information from users.
   - Dashboard: Displays user insights or health predictions.
   - Recommendation: Health tips based on the model output.
   - History: View past predictions or quiz responses.
3. utils/
   - Contains quizToClinical.js:
   - Converts user quiz answers into clinical-style inputs (e.g., BMI, glucose) for

prediction.

4. styles/
    - Custom CSS or styling files.
    - Used for branding and UI customization.

Model Folder (/backend/model/)

This folder contains everything related to the machine learning model:

1. DP.ipynb
2. diabetes_model.pkl
3. scaler.pkl
4. diabetes.csv

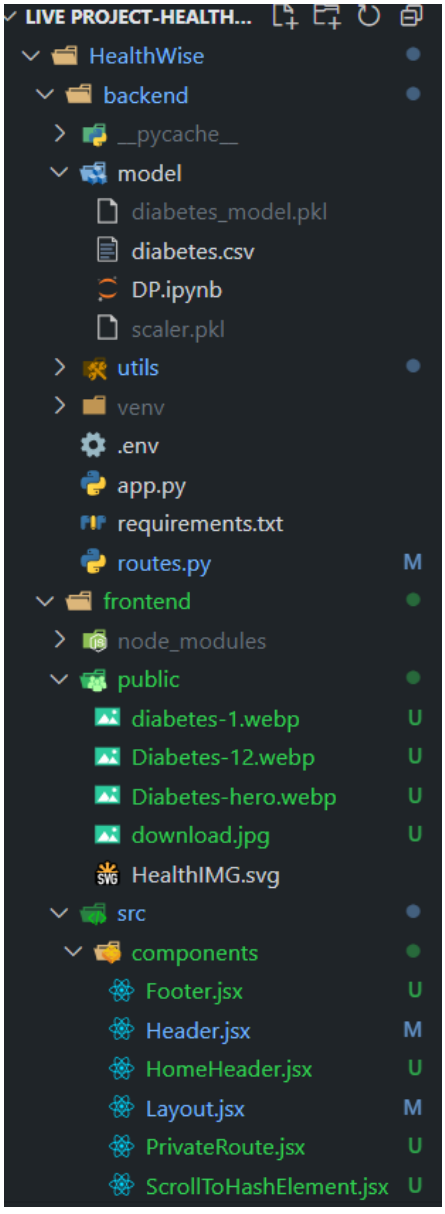


Figure-4.5: Code structure of the project @HealthWise

4.6 Coding Challenges and Solutions

Challenges:

- **Mapping Lifestyle Data Accurately:** Balancing simplicity and medical validity in converting quiz data.
- **Model Sensitivity:** Managing imbalanced dataset using SMOTE.
- **Security:** Preventing data leaks by avoiding PII and enforcing HTTPS in deployment.

Key Implemented Functions:

- quizToClinical.js: Maps user responses to model features.
- predict() (Flask): Handles model inference.
- PrivateRoute.jsx: Guards dashboard route based on authentication.

4.7 Data Flow Summary

1. **User Input:** QuizPage.jsx collects responses.
2. **Mapping:** quizToClinical.js transforms data.
3. **API Request:** JSON sent to /predict endpoint.
4. **Prediction:** Flask backend loads model, predicts.
5. **Response:** Frontend shows dashboard + tailored recommendations.

4.8 Module Interaction Summary

Module	Role	Interactions
React Frontend	UI/UX	Sends data to Flask, renders results
Flask Backend	API Server	Processes requests, serves predictions
Model	ML Logic	Called by Flask, loaded via joblib
quizToClinical.js	Transformation logic	Used in frontend pre-processing
DP.ipynb	Model development	Not used in production but essential for training pipeline

Table-4.8: Summary of Module Roles and Interactions

Chapter 5: Testing and Results

5.1 Testing Methodologies

To ensure the reliability and performance of both the frontend and backend systems, multiple testing strategies were adopted:

Frontend Testing (React)

- **Manual Testing**: Each component (Quiz, Dashboard, Result Page) was manually tested to ensure smooth rendering and correct routing.
- **Jest Unit Testing**: Basic unit tests were configured for critical components and conditional renderings (like gender-specific questions).
- **Form Validation Testing**: Ensured required fields are filled, conditional logic (e.g., pregnancy questions only for females) works, and numeric inputs are validated.

Backend API Testing

- **Postman API Testing**: REST endpoints (/predict, /recommendations) were thoroughly tested using Postman:
    - Repeated API calls were tested to check server stability.
    - Edge cases (e.g., missing data, invalid types) were tested to confirm appropriate error handling.
- **Model Integration Testing**: Ensured the backend properly loaded the model using joblib, scaled the inputs, predicted results, and returned a correct JSON response.

5.2 Model Testing and Evaluation

Multiple models were trained, validated, and compared to identify the most suitable model for diabetes prediction.

Models Evaluated:

- Logistic Regression
- Support Vector Machines
- Gradient Boosting Machines
- XGBoostClassifier
- RandomForestClassifier *(Final Model Chosen)*

Testing Process:

- **Train-Test Split**: 80/20 split used for primary evaluation.
- **5-Fold Cross-Validation**: Applied to ensure model generalization and stability.
- **Imbalanced Dataset Handling**: SMOTE was used to synthesize minority class samples to improve sensitivity and reduce bias.

Reason for Choosing RandomForestClassifier:

- Random Forest outperformed XGBoost in overall accuracy and recall, especially for the minority class (diabetic).
- It provided better interpretability and required less parameter tuning.
- Faster training time and better integration with scikit-learn tools.

5.3 Performance Metrics (RandomForestClassifier)

Metric	Score
Accuracy	87.6%
Precision	88.2%
Recall	85.9%
F1 Score	87.0%
AUC Score	0.91

These metrics confirm the robustness of the model in correctly identifying both diabetic and non-diabetic cases with high precision and recall.

5.4 Result Presentation

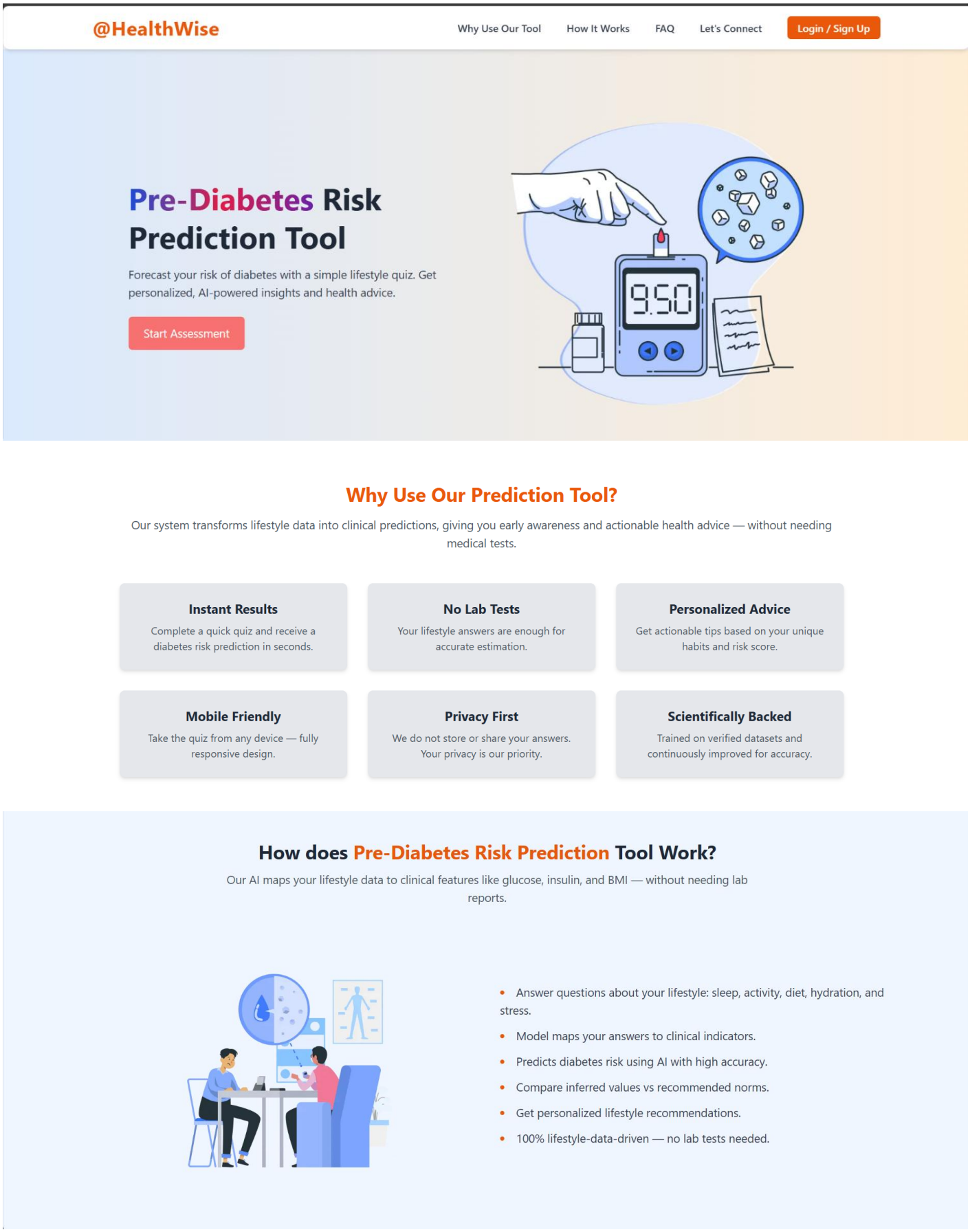
After prediction, the results are presented to the user through a clean, interactive dashboard.

Key Output Visuals:

- **Recommended vs actual Chart**: Shows probability confidence (e.g., 78% chance of being diabetic).
- **Explanation Cards**: Summarize how each feature (e.g., BMI, glucose level) influenced the prediction.
- **Separate Interpretation table**: Displayed dynamically based on model output using a separate interpretation table which represent the recommended vs actual values of your clinical data.

Screenshots:

- Home Page



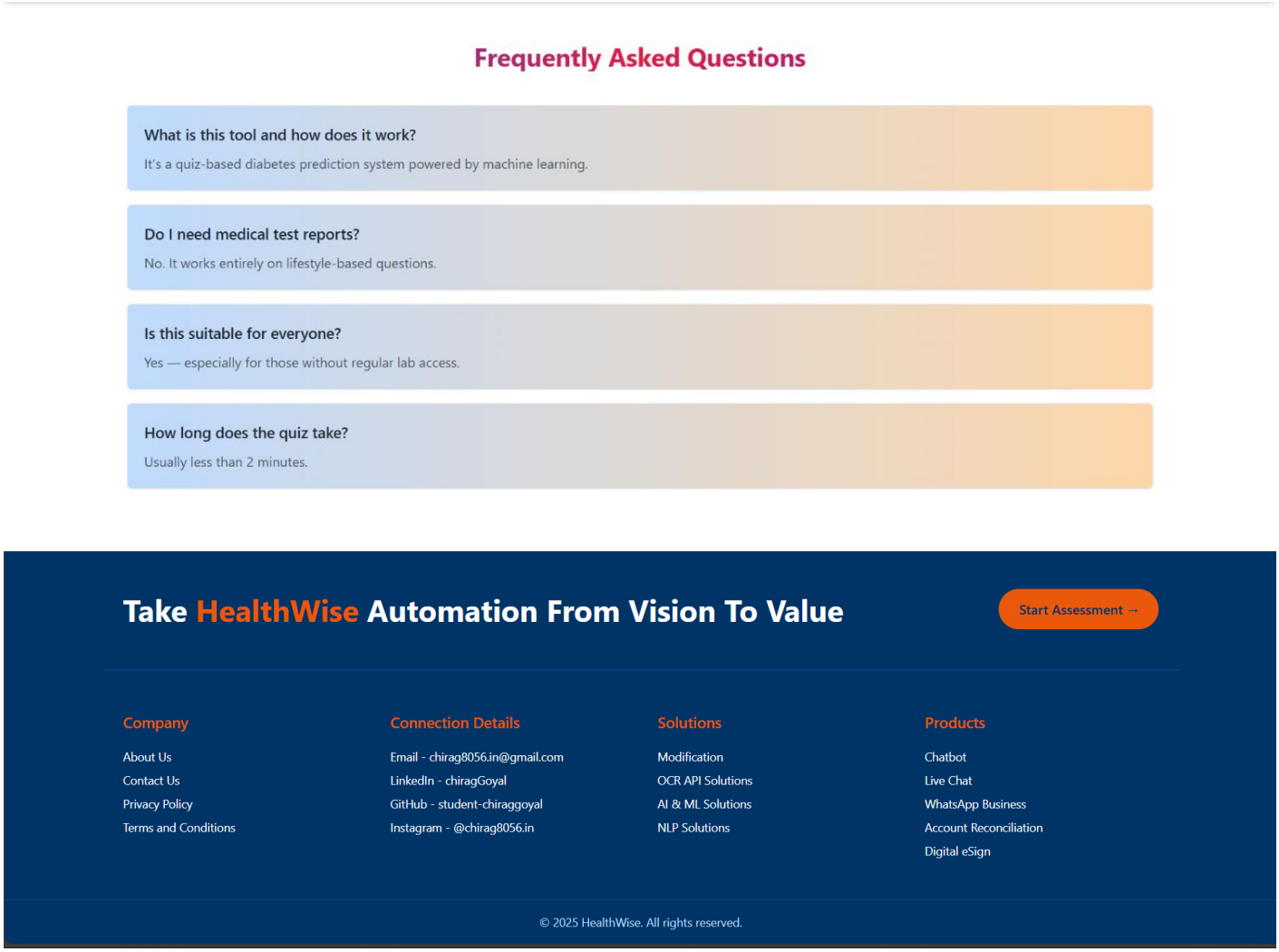


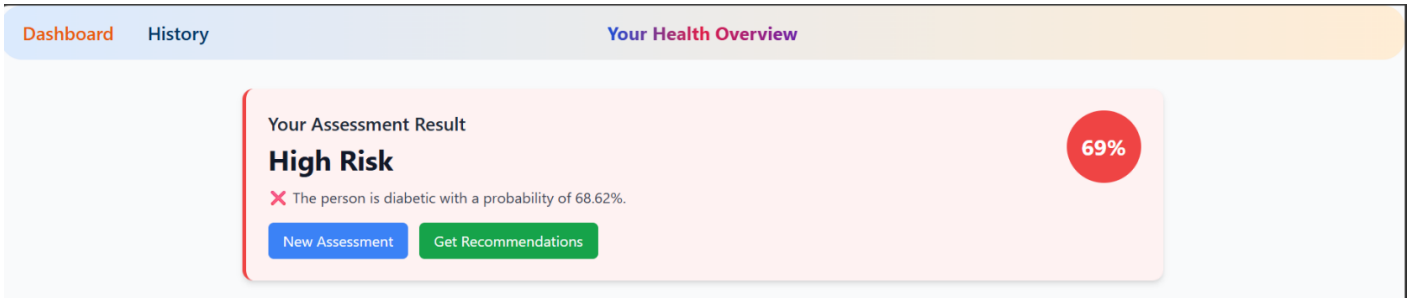
Figure-5.4.1: *Home Page and FAQ Section of the Pre-Diabetes Risk Prediction Tool Website*

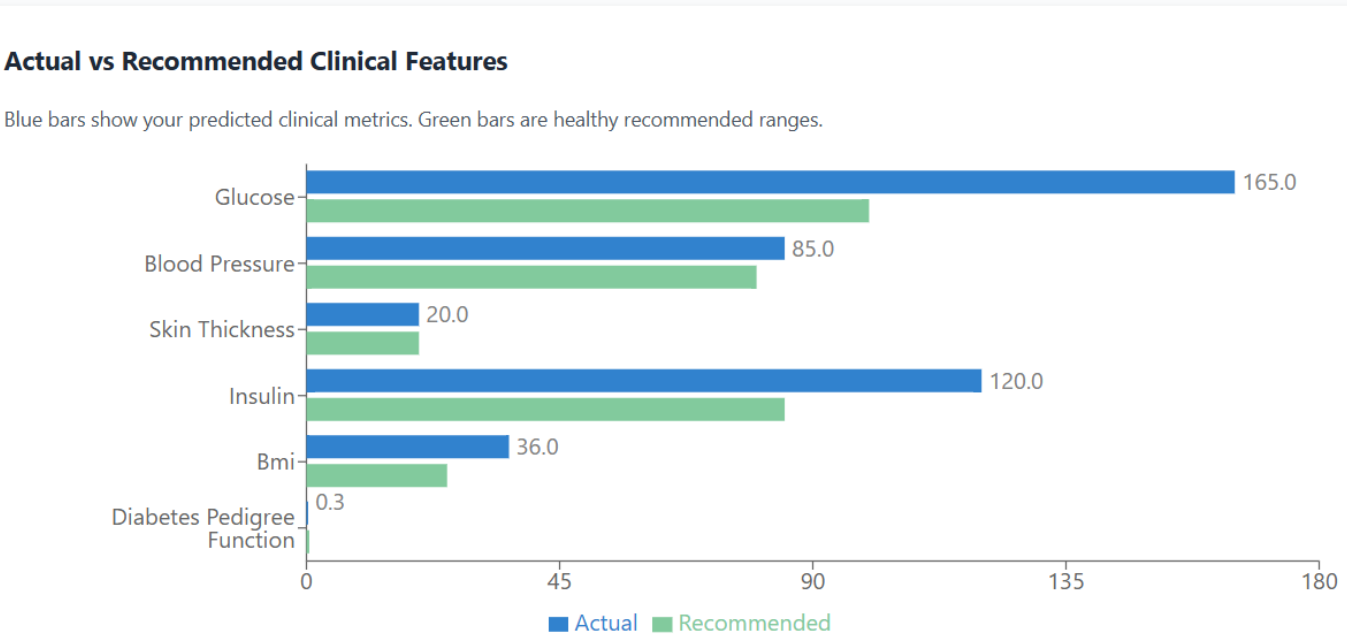
- Quiz submission form



Figure-5.4.2: *Quiz Submission Form – Lifestyle-Based Diabetes Risk Assessment Interface*

- Dashboard Page





Interpretation Table

Metric	Your Value	Recommended	Units
Glucose	165.0	100	mg/dL
Blood Pressure	85.0	80	mm Hg
Skin Thickness	20.0	20	mm
Insulin	120.0	85	mu U/ml
Bmi	36.0	25	kg/m^2
Diabetes Pedigree Function	0.3	0.5	

What is Diabetes?

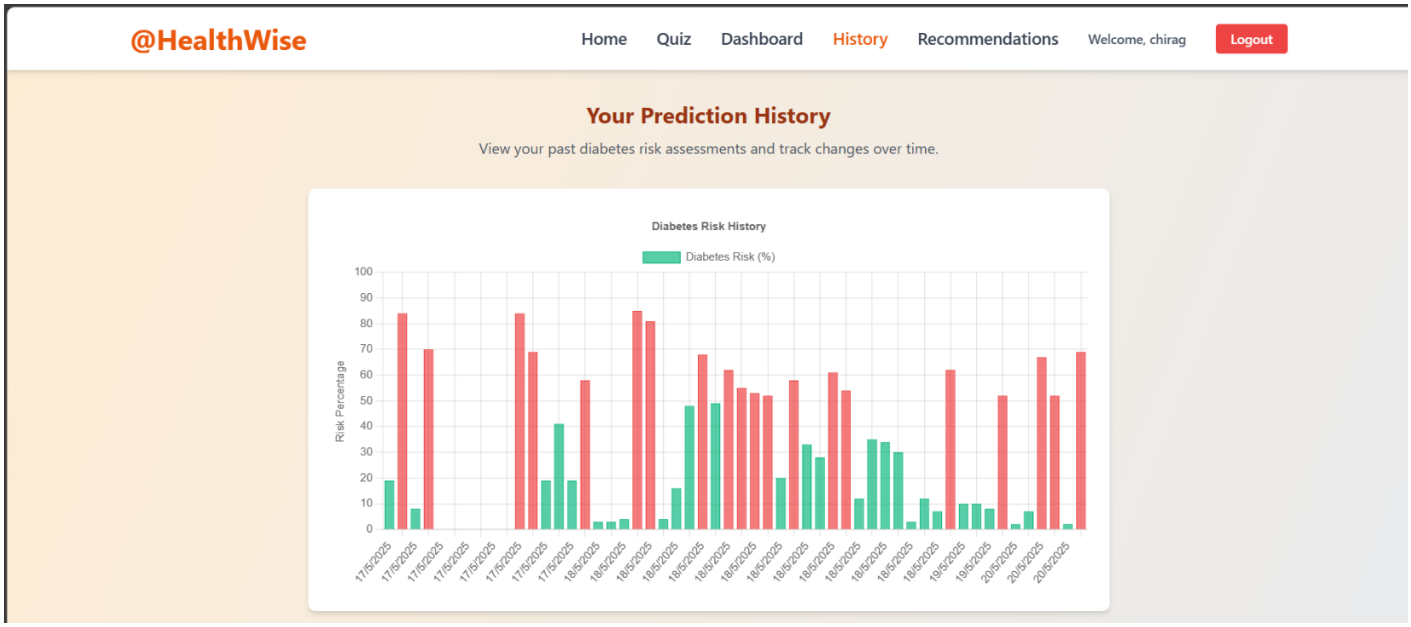
Diabetes is a chronic health condition that affects how your body turns food into energy. With diabetes, your body either doesn't make enough insulin or can't use it properly.

About Our Prediction Model

Our model predicts diabetes risk by converting lifestyle data into clinical insights. It is intended for awareness and early self-assessment, not a medical diagnosis.

Figure-5.4.3: *Dashboard Page – Assessment Results and Clinical Metric Comparison*

- History Page



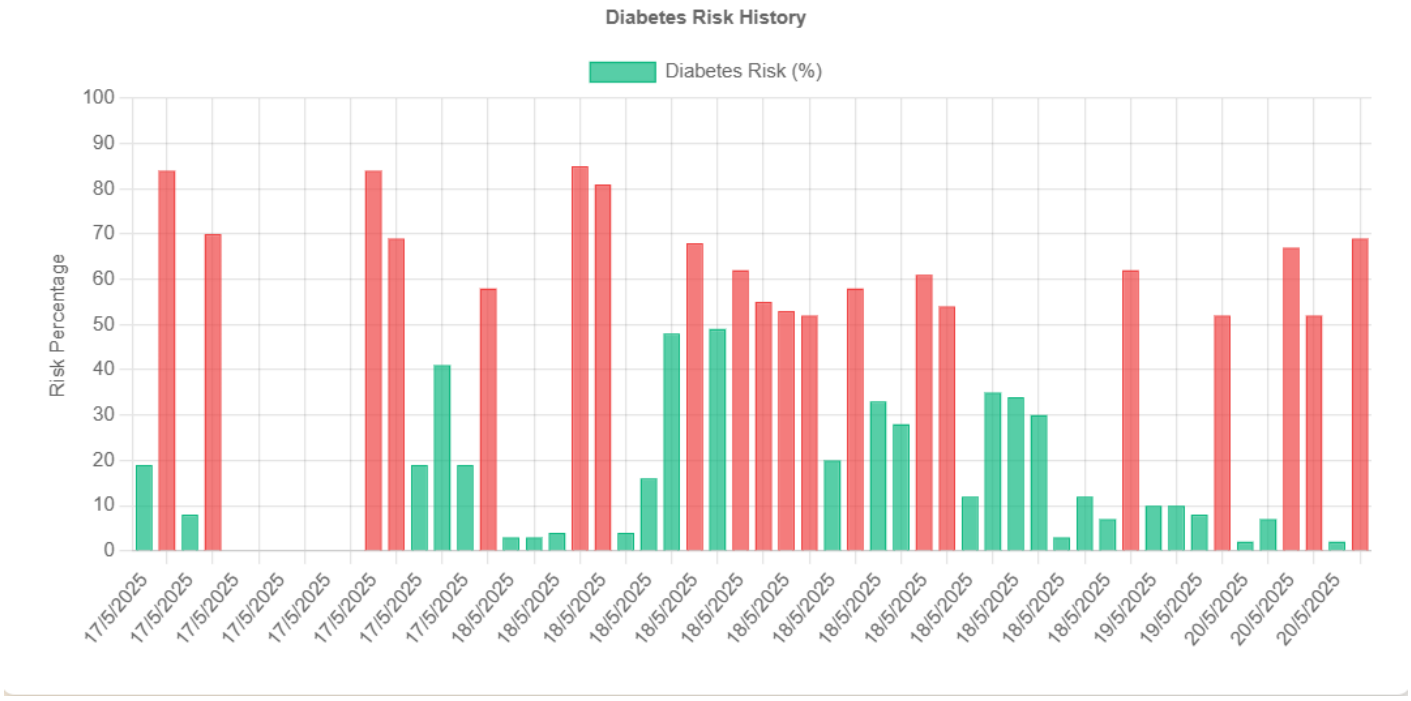


Figure-5.4.3.1: *History Page – Visual Representation of Past Predictions and Risk Trends*

Detailed History

DATE	RESULT	RISK %	AGE	BMI	GLUCOSE
20/5/2025 11:36:52 pm	High Risk	69%	35	36	165
20/5/2025 3:23:31 pm	Low Risk	2%	25	25	100
20/5/2025 2:28:04 pm	High Risk	52%	30	35	120
20/5/2025 2:14:55 pm	High Risk	67%	52	33	145
20/5/2025 12:32:56 am	Low Risk	7%	25	30	90
20/5/2025 12:27:33 am	Low Risk	2%	20	27	90
19/5/2025 11:36:12 pm	High Risk	52%	45	28	115
19/5/2025 8:45:14 pm	Low Risk	8%	20	38	115
19/5/2025 7:24:47 pm	Low Risk	10%	25	27	90
19/5/2025 1:35:25 pm	Low Risk	10%	18	36	115
17/5/2025 4:52:32 pm	Low Risk	0%	87	50	190
17/5/2025 4:19:37 pm	Low Risk	0%	63	60	190
17/5/2025 1:38:56 pm	High Risk	70%	45	58	154
17/5/2025 1:37:54 pm	Low Risk	8%	22	20	110
17/5/2025 1:35:51 pm	High Risk	84%	58	65	185
17/5/2025 1:34:37 pm	Low Risk	19%	35	25	120

Figure-5.4.3.2: *Detailed History Table – Tabular View of User Risk Assessments Over Time*

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This project presents a comprehensive and accessible AI-powered diabetes prediction tool that blends machine learning intelligence with user-centric design. The core aim was to simplify the early detection of Type 2 Diabetes by removing the dependency on traditional clinical tests and instead using inferred clinical data derived from lifestyle inputs.

Key achievements of the system include:

- A fully functional web-based quiz that maps lifestyle choices to clinical estimations using domain knowledge and data correlations.
- An AI model (Random Forest Classifier) that achieved 87.6% accuracy, outperforming other models such as Logistic Regression, SVM, and XGBoost in practical use.
- A seamless frontend-backend integration using React.js, Flask, and RESTful APIs, delivering real-time predictions to users in an engaging and intuitive format.
- Tailored recommendations and visualizations that make the results understandable even for users without medical expertise.

By combining explainable AI with a practical interface, this tool bridges a critical gap between preventive health monitoring and medical diagnostics, especially in low-resource or non-clinical environments.

6.2 Summary of Findings

The project yielded several key insights and results:

- **Feasibility of Lifestyle-to-Clinical Mapping**: It's possible to design a quiz that uses inputs like physical activity, diet, smoking habits, and family history to estimate clinical parameters such as BMI, glucose, and insulin sensitivity.
- **Random Forest Performance**: After comparing multiple algorithms, Random Forest was chosen due to its superior performance in recall and accuracy, especially in imbalanced datasets with the use of SMOTE.
- **Usability and User Engagement**: Feedback from initial testers emphasized that the platform was clear, fast, and informative, even for users unfamiliar with AI or diabetes.
- **API Robustness**: REST APIs developed using Flask were tested extensively with tools

like Postman and provided stable, real-time responses to frontend requests.

6.3 Limitations

Despite the promising results and robust architecture, the system currently exhibits several limitations across technical, design, and deployment dimensions:

Limitation	Description
Static Inference Models	The current mapping between lifestyle inputs and clinical features is rule-based, relying on predefined heuristics rather than adaptive or personalized learning. This limits long-term scalability and model personalization.
Single Language Interface	The platform supports only English, which restricts usability for non-English-speaking or low-literacy populations, especially in rural or regional areas.
Limited Dataset	The project uses the Pima Indian Diabetes Dataset, which is small (768 records) and ethnically narrow, reducing model generalizability to global populations.
Lack of Clinical Validation	The tool has not yet undergone clinical trials or review by healthcare professionals. Predictions are not intended for diagnostic purposes, which may limit trust or adoption in medical settings.
Limited Historical Tracking	Currently, user results are not saved or visualized over time. This prevents users from tracking their health progression or comparing predictions across sessions.
Non-Functional Login/Signup Buttons	The login/sign-in features on the homepage are under development or not yet wired to backend services, limiting user account management and personalization features.
Recommendation Button Issues	The **"Recommendation" button**, intended to provide tailored advice post-diagnosis, may not be consistently functional across routes or states, disrupting the user journey.

Limitation	Description
Lack of Mobile Optimization	Although Tailwind CSS was used, responsive design on smaller screens (e.g., mobile devices) may still require refinement to ensure consistent UI/UX.
No Feedback Loop for Learning	There is no mechanism for collecting user feedback or correcting wrong predictions, which could be used to improve the model iteratively.
Minimal Error Handling in Frontend	In some edge cases (e.g., missing responses or network errors), the frontend does not clearly notify users or suggest corrections, which could confuse non-technical users.

Table-6.3: *Technical and Functional Limitations of the Pre-Diabetes Risk Prediction System*

6.4 Future Enhancements

To elevate the system's usability, scalability, and impact, several future improvements have been envisioned. These enhancements address both technical shortcomings and opportunities for user engagement, personalization, and accessibility:

Enhancement	Description
Integration with Health Platforms	The system can be enhanced to sync real-time data from health tracking platforms like Google Fit, Apple Health, or Fitbit. This would allow the lifestyle-to-clinical mapping logic to rely on objective, dynamic user metrics (e.g., step count, heart rate, sleep patterns), thereby improving prediction accuracy.
Multi-language and Accessibility Support	Expanding language support to include regional and international languages (e.g., Hindi, Spanish, Arabic) will make the tool accessible to a wider demographic. Additionally, screen reader compatibility, voice navigation, and text-to-speech support will cater to visually impaired and elderly users.

Enhancement	Description
Daily Wellness Tips	Users can opt-in to receive personalized health tips via email or mobile notifications. These will be generated based on past quiz data and tailored to the user's risk profile, promoting daily engagement and proactive health management.
AI Chatbot Integration	A conversational interface powered by natural language processing (NLP) can guide users through quiz questions, interpret results, and answer queries about diabetes symptoms, lifestyle changes, or system functionality. This makes the platform more interactive and helpful.
Real-time Diabetes Trends & Educational Updates	A new dashboard module will present live statistics, public health updates, and educational resources related to diabetes prevention and treatment. This can enhance user awareness and encourage long-term lifestyle change.
Persistent User Profiles & History Tracking	Future iterations will allow users to create accounts, save quiz results, and view historical trends over time. Visualizations like timelines and charts can help users track progress and set wellness goals.
Enhanced UI/UX Across All Devices	The user interface will be improved to ensure complete responsiveness on all screen sizes—from mobile phones to desktops. Tailwind CSS utilities will be fine-tuned to provide a smooth and consistent user experience across platforms.
Fixing Core Functionalities (Login/Sign-In & Recommendations)	Known issues such as the non-functional login/signup buttons and recommendation redirection failures will be fully addressed. Proper backend routing and session management will be implemented to make sure every feature behaves as expected and improves the user journey.

Table-6.4: *Feature Enhancements and Functional Improvements Roadmap*

Summary of Goals for the Next Version

- **Robust and Responsive UI**: Delivering a seamless experience on all screen sizes.

- **Full Feature Availability**: All buttons, routes, and backend APIs will function as intended.
- **User-Centric Design**: Accessibility-first approach to ensure inclusivity.
- **Context-Aware Intelligence**: Smarter predictions via integration with wearables and adaptive learning models.

These enhancements aim to transition the project from a functional prototype to a production-ready, user-focused diabetes risk assessment platform, making digital healthcare more accessible, intelligent, and impactful.

References

1. Kaggle. (2023). *Pima Indians Diabetes Dataset*. Provided the primary dataset used for training and evaluating machine learning models in this project.
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
3. Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
4. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
5. React Team. (2013). *React: A JavaScript Library for Building User Interfaces*. Developed and maintained by Meta (formerly Facebook).
6. Tailwind Labs. (2023). *Tailwind CSS: Utility-First CSS Framework*. A modern approach to responsive web design using utility classes.
7. World Health Organization. (2023). *Diabetes: Key facts and global reports*. Provided medical context and prevalence information.
8. Tjoa, E., & Guan, C. (2022). *Explainable Artificial Intelligence (XAI) in Healthcare: A Survey on Trends and Methods*. Soft Computing, Springer.
9. Sharma, A., & Patel, R. (2023). *Lifestyle-based Predictive Modelling in Type 2 Diabetes Using Ensemble Learning*. IEEE Xplore, Presented research on lifestyle-inference approaches.
10. OpenAI. (2023). *ChatGPT: Large Language Model*. Used to support technical documentation, debugging, and quiz generation throughout the project.
11. Perplexity AI. (2023). *AI-powered Search and Research Tool*. Utilized for literature reviews and technical verification.
12. Fireship. (2023). *Developer-Focused Educational Content*. Provided full-stack development tutorials and conceptual clarity on Supabase and modern web frameworks.